



**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ»
(национальный исследовательский университет)**

УТВЕРЖДАЮ

Заведующий кафедрой 806

(№ каф.)

_____ Крылов С. С.

(подпись)

(инициалы, фамилия)

_____ 2026 г.

**ОТЧЕТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ**

по теме:

**Платформы видеоаналитики
реального времени для мониторинга и событийного
анализа в системах охранного видеонаблюдения.**

Научный руководитель _____

Гаврилов Константин

Юрьевич

доктор технических наук,

профессор

Исполнитель _____

Нурмухамедов А. Б.

Москва 2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	9
ВВЕДЕНИЕ	11
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	18
1.1 Особенности предметной области и постановка задачи	18
1.1.1 Пользовательские роли и сценарии использования	20
1.2 Обзор существующих решений и литературный обзор	21
1.2.1 Краткий литературный обзор	21
1.2.2 Обзор существующих платформ видеоаналитики	23
1.2.3 Выводы по обзору	25
1.3 Требования к разрабатываемой платформе	25
1.3.1 Функциональные требования	26
1.3.2 Нефункциональные требования	26
1.4 Обоснование выбора методов	27
1.4.1 Выбор метода обнаружения объектов	27
1.4.2 Выбор метода сопровождения объектов	29
1.4.3 Выбор метода событийного анализа	30
1.5 Сравнительный анализ альтернатив реализации платформы	31
1.5.1 Альтернативы по подсистеме захвата видеопотоков	31
1.5.2 Альтернативы по интеграционному контуру	32
1.5.3 Альтернативы по хранилищу	32
1.5.4 Архитектурные компромиссы	33
1.6 Исходные данные для разработки и экспериментальной оценки	33
1.6.1 Состав входных данных	33
1.6.2 Требования к подготовке данных	34
1.6.3 Показатели оценки качества	35
1.7 Связь требований с измеряемыми метриками и единой нотацией	36
Выводы по главе 1	37

2	ТЕОРЕТИЧЕСКИЕ ПРЕДПОСЫЛКИ, МОДЕЛИ И МЕТОДО-	39
	ЛОГИЯ	
2.1	Формальная модель конвейера обработки	39
2.2	Обоснование архитектуры и технологического стека	40
2.2.1	Архитектурный паттерн	41
2.2.2	Алгоритмическое ядро: детекция и сопровождение . .	42
2.2.3	Веб-фреймворк управляющего сервиса	43
2.2.4	Система управления базами данных	44
2.2.5	Брокер сообщений	44
2.2.6	Подсистема видеозахвата	45
2.2.7	Наблюдаемость и оркестрация	45
2.2.8	Сводный технологический стек	46
2.2.9	Логическая модель работы платформы	49
2.3	Модель детекции объектов и фильтрации классов	50
2.3.1	Выход детектора и базовые обозначения	50
2.3.2	Фильтрация целевых классов	51
2.3.3	Оценка качества детекции	51
2.4	Модель сопровождения объектов	52
2.4.1	Состояние трека и прогноз	52
2.4.2	Ассоциация детекций и траекторий	52
2.4.3	Устойчивость идентификаторов и fallback-логика . . .	53
2.5	Правиловая модель событий и дедупликация	53
2.5.1	Событие как предикат над траекторией	53
2.5.2	Правило <code>zone_enter</code>	54
2.5.3	Правило <code>line_cross</code>	54
2.5.4	Правило <code>dwell_time</code>	54
2.5.5	Модель <code>dedup_key</code>	54
2.6	Модель надежной доставки: исходящие сообщения, повторы и очередь недоставленных сообщений	55
2.6.1	Транзакционная фиксация события	55
2.6.2	Повторные попытки публикации	55
2.6.3	Модель потребителя и очереди недоставленных сооб- щений	55
2.7	Оценка вычислительной сложности и ресурсной нагрузки . .	56

2.7.1	Сложность стадий конвейера	56
2.7.2	Задержка сквозного контура	57
2.7.3	Ресурсные ограничения на одном узле	57
2.8	Псевдокод целевого алгоритма	58
2.8.1	Алгоритм обработки одной камеры	58
2.8.2	Алгоритм работы публикатора исходящих сообщений	58
2.8.3	Алгоритм доставки HTTP-получателю	59
2.9	Параметризация и калибровка модели	59
2.9.1	Набор настраиваемых параметров	59
2.9.2	Принцип калибровки	59
2.9.3	Ограничения валидности модели	60
2.10	Метрики качества подсистем и их агрегация	60
2.10.1	Метрики детекции	60
2.10.2	Метрики сопровождения	61
2.10.3	Метрики событийного уровня	61
2.10.4	Метрики надежности интеграции	61
2.10.5	Сквозная агрегированная оценка	62
2.11	Формализация ограничений и критериев приемки	62
2.11.1	Ограничения по времени	62
2.11.2	Ограничения по надежности	62
2.11.3	Ограничения по безопасности	63
2.11.4	Критерий архитектурной приемки	63
	Выводы по главе 2	63
3	ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	65
3.1	Архитектура сервисов и инфраструктурный контур	65
3.2	Реализация программного интерфейса (API), JWT/RBAC и контрактов	66
3.2.1	Технологический стек программного интерфейса	66
3.2.2	Модель пользователей и ролей	66
3.2.3	Сценарий JWT-аутентификации	67
3.2.4	Контракты REST API	67
3.3	Реализация analytics-worker	68
3.3.1	Захват и декодирование видеопотока	68
3.3.2	Детекция и сопровождение объектов	68

3.3.3	Правиловой движок и генерация событий	69
3.4	Реализация integration-worker: RabbitMQ, повторы и очередь недоставленных сообщений	70
3.4.1	Топология RabbitMQ	70
3.4.2	Публикатор исходящих сообщений	70
3.4.3	Доставка HTTP-получателю и журнал попыток	71
3.5	Мониторинг, метрики и эксплуатационная наблюдаемость	72
3.6	Развертывание и воспроизводимость	72
3.6.1	Docker Compose профиль	72
3.6.2	Makefile и автоматизация	72
3.7	Реализация операторского интерфейса (UI)	73
3.7.1	Структура клиентского приложения	73
3.7.2	Сценарий JWT-аутентификации на стороне интерфейса	73
3.7.3	Ролевая видимость действий	73
3.8	Структура данных и миграции	76
3.8.1	Логическая схема БД	76
3.8.2	Подход к миграциям	77
3.8.3	Индексы и производительность запросов	77
3.9	Тестирование, верификация и критерии качества	78
3.9.1	Стратегия тестирования	78
3.9.2	Обязательные проверки до стендового прогона	78
3.9.3	Критерии приемки реализации	79
3.10	Эксплуатационные регламенты и ограничения текущего контура	79
3.10.1	Операционный регламент	79
3.10.2	Известные ограничения	79
3.11	Сценарии межсервисного взаимодействия	80
3.11.1	Сценарий «создание конфигурации оператором»	80
3.11.2	Сценарий «событие от кадра до HTTP-получателя»	80
3.11.3	Сценарий «временная недоступность брокера»	81
3.12	Конфигурационное управление и версионирование контрактов	81
3.12.1	Версионирование API	81
3.12.2	Версионирование сообщения события	81
3.12.3	Управление конфигурацией правил	81
3.13	Инженерные аспекты производительности	82

3.13.1	Горизонт и вертикаль масштабирования	82
3.13.2	Параметры настройки фоновых обработчиков	82
3.13.3	Управление обратным давлением	82
3.14	Операционная диагностика и сопровождение	83
3.14.1	План диагностики при деградации	83
3.14.2	Набор базовых предупреждений	83
	Выводы по главе 3	83
4	АПРОБАЦИЯ И АНАЛИЗ РЕЗУЛЬТАТОВ	85
4.1	Цели, стенд и границы апробации	85
4.2	Экспериментальные данные и артефакты	86
4.3	Метрики и порядок расчета	89
4.4	Результаты приемочного прогона	90
4.5	Интерпретация результатов	94
4.6	Ограничения апробации и угрозы валидности	96
4.7	Практические рекомендации по итогам апробации	96
	Выводы по главе 4	97
	ЗАКЛЮЧЕНИЕ	98
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	102
	ПРИЛОЖЕНИЕ А. КОНТРАКТ СОБЫТИЯ И КОНЕЧНЫЕ ТОЧКИ	
	API	106
A.1	Контракт события RabbitMQ	106
A.2	Ключевые конечные точки REST API	106
A.3	Пример dedup_key	107
A.4	Примеры запросов и ответов API	107
A.5	Минимальная JSON-схема события (фрагмент)	111
	ПРИЛОЖЕНИЕ Б. ТОПОЛОГИЯ РАЗВЕРТЫВАНИЯ И ПРОФИЛИ	
	ЗАПУСКА	113
B.1	Минимальный compose-контур	113
B.2	Профили запуска	113
B.3	Операционные команды	114
B.4	Рекомендации по воспроизводимости	114

Б.5 Переменные окружения (пример)	114
Б.6 Фрагмент docker-compose (концептуально)	115
Б.7 Профиль дальнейшего аппаратного ускорения	116
ПРИЛОЖЕНИЕ В. ПРОТОКОЛЫ ЭКСПЕРИМЕНТОВ	117
В.1 Карточка приемочного прогона	117
В.2 Протокол метрик прогона	118
В.3 Протокол отказных сценариев	118
В.4 Сохраненные артефакты	119
В.5 Чек-лист функциональных сценариев	120
В.6 Чек-лист надежности и безопасности	121
В.7 Итоговый акт стендового прогона	121

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API	Application Programming Interface, программный интерфейс приложения
CPU	Central Processing Unit, центральный процессор
CRUD	Create, Read, Update, Delete, базовые операции управления данными
Docker	Платформа контейнеризации приложений
E2E	End-to-End, сквозной сценарий проверки
FFmpeg	Набор библиотек и утилит для обработки медиа
FPS	Frames Per Second, кадров в секунду
GPU	Graphics Processing Unit, графический процессор
HA	High Availability, высокая доступность
HTTP	HyperText Transfer Protocol
IoU	Intersection over Union, метрика перекрытия прямоугольников
JSON	JavaScript Object Notation, текстовый формат обмена структурированными данными
JWT	JSON Web Token, веб-маркер JSON для передачи утверждений об идентификации
MOT	Multi-Object Tracking, многообъектное сопровождение
NMS	Non-Maximum Suppression, подавление немаксимумов
RBAC	Role-Based Access Control, ролевая модель управления доступом
ReID	Re-identification, повторная идентификация объекта
REST	Representational State Transfer
RTSP	Real Time Streaming Protocol
SLA	Service Level Agreement
SQL	Structured Query Language
UI	User Interface, пользовательский интерфейс
UUID	Universally Unique Identifier

YOLO

You Only Look Once, семейство одноэтапных детекторов

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Видеоаналитика реального времени - совокупность методов и программных средств, обеспечивающих анализ видеопотока с ограниченной задержкой и выдачей результатов, пригодных для оперативного реагирования.

Детекция объекта - результат локализации и классификации объекта в кадре, содержащий ограничивающий прямоугольник, класс и оценку достоверности.

Сопровождение объекта - поддержание устойчивой траектории объекта между последовательными кадрами видеопотока.

Трек (траектория) - последовательность состояний одного и того же физического объекта в ряде кадров, объединенная единым устойчивым идентификатором.

Инференс модели - применение обученной модели к входному кадру для получения детекций или иных выходных признаков.

Событие - интерпретируемый факт, сформированный правилами движком на основе траектории, пространственной геометрии сцены и временных условий.

Зона контроля - заданная область кадра, используемая для формализации ограничений и условий генерации событий.

Виртуальная линия - ориентированный отрезок (или ломаная) в координатах кадра, пересечение которого траекторией используется как триггер события.

Транзакционная таблица исходящих сообщений (transactional outbox) - архитектурный шаблон надежной доставки, при котором событие и запись о его последующей публикации фиксируются в одной транзакции в базе данных.

Идемпотентность доставки - свойство обработчика, при котором повторная обработка одного и того же сообщения не приводит к искажению итогового состояния внешней системы.

НТТР-получатель - внешняя система или тестовый сервис, принимающий сведения о событиях платформы по НТТР-запросам.

Очередь недоставленных сообщений - очередь брокера сообщений, в

которую помещаются сообщения после исчерпания допустимого числа попыток доставки.

ВВЕДЕНИЕ

Развитие систем охранного видеонаблюдения сопровождается постоянным ростом числа камер, объема поступающих видеоданных и требований к скорости реагирования на потенциальные инциденты. Современные распределенные объекты охраны (промышленные площадки, транспортные узлы, периметры закрытых территорий, объекты критической информационной инфраструктуры) ежедневно генерируют значительный объем видеоматериала, существенная часть которого требует оперативной интерпретации в режиме реального времени. В таких условиях ручной мониторинг видеопотоков становится организационно ограниченным, а практическая ценность платформы определяется сочетанием автоматического обнаружения, сопровождения объектов и надежной событийной доставки [1—3].

Вместе с тем требования нормативных и отраслевых документов, регулирующих проектирование, испытания и эксплуатацию систем видеонаблюдения, формируют дополнительные ограничения на полноту фиксации событий, время их доставки во внешние системы и устойчивость к отказам отдельных компонентов [4; 5]. Это закономерно повышает потребность в платформах видеоаналитики реального времени, способных автоматически обнаруживать объекты, сопровождать их в последовательности кадров, интерпретировать наблюдаемую ситуацию и надежно передавать сведения о событиях внешним потребителям. Особую значимость приобретают сценарии, связанные с обнаружением малозаметных и динамичных объектов, в том числе беспилотных летательных аппаратов, которые на ряде объектов охраны рассматриваются как самостоятельный класс угроз [6].

Актуальность темы исследования определяется как научными, так и практическими факторами. С научной точки зрения существенный интерес представляет комплексное согласование методов обнаружения объектов, их сопровождения и событийного анализа с архитектурными механизмами, обеспечивающими работу системы в условиях ограниченной задержки и неустойчивых внешних интеграций. Для целей настоящей работы недостаточно рассматривать только качество детектора или только надежность программной инфраструктуры: значимым является их совместное проекти-

рование и экспериментальная проверка в едином контуре. С практической точки зрения востребованы решения, позволяющие сократить нагрузку на операторов служб безопасности, уменьшить время реакции на инциденты, повысить надежность мониторинга, обеспечить совместимость с действующими системами оповещения и управления и снизить риск потери событий при сбоях транспортного слоя [3; 7].

Цель работы состоит в разработке и экспериментальной проверке на исследовательско-экспериментальном стенде платформы видеоаналитики реального времени для мониторинга охраняемых зон и событийного анализа, обеспечивающей обнаружение и сопровождение объектов, формирование интерпретируемых событий и надежную доставку сведений о событиях внешним потребителям.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) проанализировать предметную область охранного видеонаблюдения, существующие подходы к построению платформ видеоаналитики и сформировать функциональные и нефункциональные требования к разрабатываемому решению;
- 2) обосновать выбор методов обнаружения объектов, сопровождения объектов и правилowego событийного анализа для целевых сценариев видеонаблюдения, включая демонстрационный сценарий обнаружения беспилотных летательных аппаратов;
- 3) разработать модель, архитектуру и алгоритмический конвейер платформы, объединяющие прием видеопотока, обнаружение, сопровождение, формирование событий и асинхронную доставку сведений о событиях внешним потребителям;
- 4) реализовать программную платформу с компонентами управления, аналитической обработки, интеграционной доставки, разграничения доступа, наблюдаемости и воспроизводимого развертывания;
- 5) провести стендовую апробацию платформы по интеграционным, отказным, ролевым и демонстрационным видеосценариям с фиксацией границ доказанности полученных результатов;
- 6) определить ограничения текущей реализации, условия ее применимости и направления дальнейшего развития для перехода от

исследовательско-экспериментального стенда к промышленному применению.

Объектом исследования являются процессы автоматизированного мониторинга видеопотоков в системах охранного видеонаблюдения.

Предмет исследования составляют методы, модели и архитектурные решения платформ видеоаналитики реального времени, обеспечивающие обнаружение и сопровождение объектов, формирование событий и надежную доставку сведений о событиях при ограничениях по задержке обработки, отказоустойчивости и безопасности доступа.

Теоретическую и методическую основу исследования составляют:

- методы компьютерного зрения и машинного обучения для задач обнаружения и сопровождения объектов в видеопотоках [1; 2; 8];
- методы цифровой обработки видео, включая декодирование потоков и работу с временными метками [9; 10];
- методы оценки качества алгоритмов обнаружения и сопровождения, в том числе на основе методик MOT-Challenge и метрик MOTMetrics [11; 12];
- методы проектирования распределенных программных систем и надежной асинхронной доставки сообщений, в том числе шаблон транзакционной таблицы исходящих сообщений и политики повторной доставки [3; 13; 14];
- методы экспериментальной оценки характеристик систем реального времени, включая измерение сквозной задержки, контроль устойчивости при отказах и сбор операционных метрик [5; 7; 15];
- подходы к разграничению доступа и аудиту действий пользователей в информационных системах [16];
- требования и подходы к оформлению результатов научно-технических исследований, в том числе ГОСТ 7.32-2017 [17], ГОСТ Р 7.0.5-2008 [18] и сопутствующие нормативные документы [19—21].

Научная новизна работы заключается в следующем:

- 1) предложен и стендово проверен архитектурно-алгоритмический подход к построению платформы видеоаналитики реального времени для задач охранного видеонаблюдения, обеспечивающий согласованное

решение задач обнаружения, сопровождения, событийной интерпретации и надежной доставки сведений о событиях в едином программном контуре;

- 2) уточнена модель сквозной задержки от объекта к событию ($L_{\text{object} \rightarrow \text{event}}$) с разделением на этапы декодирования, инференса, сопровождения, применения правил и асинхронной публикации, что позволяет локализовать узкие места без изменения внешних контрактов системы;
- 3) сформулированы и реализованы инженерные принципы интеграции аналитического ядра с внешними потребителями событий через шаблон транзакционной таблицы исходящих сообщений, брокер сообщений и контролируемую политику повторов, обеспечивающие сохранность доменных событий в условиях кратковременных отказов внешних компонентов;
- 4) разработан набор эксплуатационных проверок (интеграционные сценарии доставки, отказные сценарии RabbitMQ и HTTP-получателя, сценарии RBAC/JWT), пригодных для регулярного контроля архитектурных свойств платформы между релизами.

Положения, выносимые на защиту:

- 1) архитектурно-алгоритмический подход к построению платформы видеоаналитики реального времени, разделяющий контуры управления и обработки данных и обеспечивающий совместное решение задач обнаружения, сопровождения и событийного анализа объектов;
- 2) модель сквозной задержки и методика ее декомпозиции, позволяющая обоснованно выбирать направления оптимизации горячего пути обработки без нарушения внешних контрактов системы;
- 3) инженерное решение надежной асинхронной доставки доменных событий на основе шаблона транзакционной таблицы исходящих сообщений и контролируемых повторов с маршрутизацией в очередь недо-ставленных сообщений;
- 4) методика экспериментальной приемки платформы по совокупности интеграционных, отказных и ролевых сценариев проверки безопасности с фиксацией ключевых эксплуатационных метрик.

Практическая значимость работы состоит в возможности применения разработанной платформы для повышения оперативности выявления инцидентов, снижения нагрузки на операторов служб безопасности и интеграции интеллектуальной видеоаналитики с действующими системами оповещения и управления безопасностью. Результаты исследования могут быть использованы при создании и модернизации систем мониторинга охраняемых зон на объектах с повышенными требованиями к непрерывности наблюдения и скорости реакции, а также при подготовке технических заданий и регламентов эксплуатации платформ видеоаналитики, в том числе в части требований к надежности доставки событий и наблюдаемости.

Методология и методы исследования. В работе применен системный подход к проектированию программно-алгоритмических систем реального времени. Использовались методы анализа требований, проектирования архитектуры распределенных систем, экспериментального тестирования и сбора эксплуатационных метрик. Программная реализация выполнена с применением технологий FastAPI, PostgreSQL, RabbitMQ, Docker Compose, Prometheus и Grafana [7; 15; 22—25]. Алгоритмическая часть основана на одноэтапных детекторах семейства YOLO и модулях сопровождения семейства SORT/DeepSORT с дополнительной обработкой результатов в правилочном событийном движке [1; 2; 26].

Апробация результатов работы. Работоспособность и устойчивость платформы подтверждены автоматизированным приемочным прогоном интеграционного контура на CPU-профиле, регрессионными тестами API и фоновых обработчиков, а также демонстрационным сценарием обнаружения беспилотных летательных аппаратов. В рамках апробации зафиксированы:

- прохождение синтетически зафиксированных событий `zone_enter`, `line_cross` и `dwelling_time` через транзакционную таблицу исходящих сообщений, брокер сообщений и внешний HTTP-получатель;
- демонстрационный путь обработки БПЛА-видео от обученной модели до траекторий, событий и операторского интерфейса;
- устойчивость интеграционного контура при кратковременной недоступности RabbitMQ и HTTP-получателя, отсутствие потерь событий и завершение доставки после восстановления внешних компонентов;
- соблюдение ролевой модели доступа `admin/operator/viewer` в

- сценариях управления и чтения данных;
- инструментальная готовность к сбору операционных метрик средствами Prometheus и их визуализации в Grafana; статистически устойчивая p95-оценка полного пути от объекта к событию вынесена в последующий длительный стендовый прогон.

Основной проверочный артефакт апробации -- сохраненный JSON-файл приемочного прогона от 13.05.2026; он используется в работе как источник статусов, счетчиков доставки и времен восстановления, а не как иллюстративное приложение. Демонстрационные материалы БПЛА-профиля используются отдельно для подтверждения связности видеоконтура. Сценарий исчерпания повторов и перехода в очередь недоставленных сообщений подтвержден отдельной проверкой логики фонового обработчика и не смешивается с результатами RUN-2026-05-13-A1.

Таким образом, в работе не отождествляются три уровня проверки: приемочный прогон интеграционного контура, демонстрационный видеосценарий и промышленная многокамерная оценка задержки. Количественные выводы формулируются только в пределах фактически сохраненных артефактов.

Подготовлен экспериментальный протокол, предназначенный для повторения CPU-проверок, фиксации отказных сценариев и подготовки публикации по итогам исследования. Программная реализация платформы оформлена в виде набора микросервисов с воспроизводимым стендом развертывания.

Структура и объем работы. Работа состоит из введения, четырех глав, заключения, списка использованных источников и приложений. Во введении обоснована актуальность темы, сформулированы цель, задачи, объект, предмет и научная новизна работы, изложена методологическая основа и приведены сведения об апробации. В первой главе выполнен анализ предметной области, рассмотрены существующие подходы к построению платформ охранной видеоаналитики и сформулированы требования к разрабатываемому решению. Вторая глава посвящена теоретическим предпосылкам, моделям и методологии: рассматриваются логика работы платформы, ее архитектура и используемый технологический стек, а также ключевые алгоритмы и расчетные соотношения. В третьей главе приведена программная реализация

платформы, включая API, фоновые обработчики, пользовательский интерфейс (UI), контракты, листинги ключевых фрагментов кода и эксплуатационные регламенты. В четвертой главе представлены результаты апробации, таблицы приемочного прогона, демонстрационный БПЛА-профиль, графики обучения детектора и ограничения текущих измерений. В заключении систематизированы основные выводы и обозначены перспективы дальнейшего развития.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Особенности предметной области и постановка задачи

Системы охранного видеонаблюдения охраняемых зон функционируют в условиях непрерывного поступления видеоданных, неоднородности сцен наблюдения и высоких требований к времени реакции на инциденты. В отличие от задач автономного анализа отдельных изображений, в рассматриваемой предметной области необходимо не только обнаружить объект в одном кадре, но и обеспечить его устойчивое сопровождение во времени, интерпретировать поведение в контексте контролируемой зоны и сформировать событие, пригодное для дальнейшего использования внешними подсистемами безопасности [1—3].

Для платформы видеоаналитики реального времени характерны следующие особенности:

- многоканальность наблюдения, при которой одновременно обрабатывается несколько видеопотоков от территориально распределенных камер;
- изменчивость условий съемки, обусловленная временем суток, погодными факторами, различиями освещенности и появлением помех в кадре;
- наличие объектов разных масштабов и скоростей движения, включая людей, транспортные средства и малогабаритные динамичные объекты, в том числе БПЛА;
- необходимость различать отдельные факты появления объектов и значимые для оператора события, например пересечение рубежа, проникновение в запретную зону, длительное нахождение в области контроля или появление объекта заданного класса;
- наличие ограничений по вычислительным ресурсам, пропускной способности каналов передачи данных и допустимой задержке выдачи результата.

С учетом указанных особенностей задачу разработки платформы можно представить следующим образом. Пусть на вход системы поступает мно-

$$\mathcal{V} = \{V_i\}_{i=1}^N, \quad (1)$$

где V_i — видеопоток камеры i ;

x_t^i — последовательность кадров потока V_i ;

m_i — метаданные камеры;

c_i — конфигурационные параметры: зоны, рубежи, классы и правила генерации событий.

Для каждого кадра x_t^i необходимо определить множество детекций

$$\mathcal{D}_t^i = \{d_k\}, \quad (2)$$

где d_k — элемент множества детекций, содержащий положение объекта в кадре, его класс и оценку достоверности.

На основе последовательности детекций необходимо формировать множество траекторий

$$\mathcal{T}_t^i = \{\tau_j\}, \quad (3)$$

сохраняющих идентичность объекта на протяжении времени наблюдения. Далее по множеству траекторий, правилам предметной области и описаниям зон контроля требуется сформировать множество событий

$$\mathcal{E}_t^i = \{e_r\}, \quad (4)$$

каждое из которых характеризуется типом события, временем возникновения, привязкой к камере, связанными объектами и уровнем приоритета.

Таким образом, постановка задачи включает четыре взаимосвязанных этапа:

- 1) прием и предварительную обработку видеопотоков;
- 2) обнаружение и классификацию объектов в отдельных кадрах;
- 3) сопровождение объектов между кадрами с сохранением идентификаторов;
- 4) событийный анализ траекторий и формирование сведений о событиях для внешних потребителей.

Результатом работы платформы должны являться не только детекции и траектории, но и структурированные сообщения о событиях, пригодные

для передачи в операторские интерфейсы, журналы регистрации, системы оповещения и другие подсистемы безопасности. При этом система должна работать в режиме, близком к реальному времени, обеспечивая приемлемый баланс между точностью аналитики, устойчивостью сопровождения, скоростью обработки и надежностью доставки событий.

1.1.1 Пользовательские роли и сценарии использования

Помимо технической постановки задачи, для разработки платформы важно зафиксировать целевые роли пользователей и типовые сценарии их работы с системой. Это позволяет уже на этапе анализа предметной области согласовать функциональные требования с реальной деятельностью операторов и администраторов.

В платформе выделены три пользовательские роли:

- *Оператор охраны* (*operator*) - сотрудник, осуществляющий непрерывный мониторинг охраняемых зон, реагирующий на поступающие тревоги и регистрирующий результаты обработки инцидентов;
- *Администратор системы* (*admin*) - сотрудник, отвечающий за конфигурацию камер, описание зон, настройку правил событийного анализа и контроль учетных записей;
- *Аналитик безопасности* (*viewer*) - сотрудник, работающий с журналом событий и аналитическими отчетами без права изменения конфигурации.

Карта пути пользователя (Customer Journey Map, CJM) для роли оператора охраны включает следующие основные этапы:

- 1) *Начало смены.* Оператор выполняет вход в систему, проверяет состояние камер и активных правил, убеждается в исправности интеграционного контура (доставка тестовых событий).
- 2) *Непрерывный мониторинг.* На рабочем месте отображаются текущие состояния камер, сводный поток событий и индикаторы операционных метрик. Оператор контролирует ход обработки и при необходимости фильтрует поток по типу события или зоне.
- 3) *Реакция на событие.* При поступлении тревоги оператор открывает карточку события, оценивает контекст (тип правила, камера, зона, объ-

ект, оценка достоверности), при необходимости просматривает связанный фрагмент видео и фиксирует результат обработки.

- 4) *Эскалация и взаимодействие.* В случае подтверждения инцидента событие передается во внешние подсистемы оповещения и управления безопасностью; ход эскалации сохраняется в журнале для последующего разбора.
- 5) *Завершение смены.* Оператор формирует сводку обработанных событий, передает изменения сменщику и фиксирует наблюдения по работе платформы (сбои, ложные срабатывания, замечания к настройкам правил).

Аналогичные карты составлены для администратора системы (управление камерами, зонами, правилами и учетными записями) и аналитика безопасности (работа с историей событий, формирование отчетов). Совокупность ролей и сценариев непосредственно используется далее при формировании функциональных и нефункциональных требований к платформе.

1.2 Обзор существующих решений и литературный обзор

Согласно методическим рекомендациям по содержанию первой главы выпускной квалификационной работы, на этапе анализа предметной области необходимо рассмотреть существующие подходы и программные платформы, выявить их сильные и слабые стороны и тем самым обосновать актуальность собственной разработки.

1.2.1 Краткий литературный обзор

В научной литературе задачи обнаружения и сопровождения объектов в видеопотоках рассматриваются в значительном числе работ. Алгоритмы одноэтапной детекции класса YOLO впервые предложены в работе [1] и в дальнейшем развиты в семействе моделей различной вычислительной сложности [27; 28], что обеспечило их широкое практическое применение для потоковой обработки видео. Для предметной области БПЛА актуальны специализированные наборы данных и эталонные наборы испытаний: например, UAVDB фиксирует проблему малых размеров объекта, вариативности масштаба и необходимости слабоконтролируемой разметки для высокоразрешающих сцен наблюдения [6]. Двухэтапные методы и трансформерные архи-

текстуры обеспечивают более высокое качество распознавания на сложных сценах, однако требуют существенно больших вычислительных ресурсов и уступают одноэтапным подходам по показателю задержки.

В области сопровождения объектов в течение последних лет получили распространение методы сопровождения по результатам обнаружения. Алгоритм SORT [26] показал, что комбинация фильтра Калмана [8] и венгерского алгоритма [29] позволяет получать высокую скорость сопровождения при умеренной точности на простых сценах. Дальнейшее развитие этого направления привело к алгоритму DeepSORT [2], дополняющему кинематическое сопоставление признаками внешнего вида и существенно снижающему число переключений идентификаторов. Для оценки качества сопровождения используются методики MOT-Challenge и метрики MOTMetrics [11; 12], а в качестве типовой выборки для предобучения моделей детекции - набор данных COCO [30].

Построение событийно-ориентированных платформ и надежной асинхронной интеграции хорошо освещено в инженерной литературе. В монографии [13] и серии онлайн-материалов [3; 14] систематизированы шаблоны микросервисной архитектуры, включая шаблон транзакционной таблицы исходящих сообщений и шаблоны контролируемой повторной доставки. Документация используемых компонентов [7; 15; 22—25; 31; 32] формирует справочную базу для практической реализации платформы.

Нормативную и методическую основу оформления и экспериментальной оценки результатов задают документы системы СИБИД, в том числе ГОСТ 7.32-2017 [17], ГОСТ Р 7.0.5-2008 [18] и связанные стандарты библиографического описания [19—21]. Для прикладного контура видеонаблюдения также учитываются актуальные международные рекомендации по проектированию, вводу в эксплуатацию и испытанию интеллектуальной видеоаналитики [4; 5], а для контура безопасности - контрольные семейства доступа и аудита [16]. Сочетание академической литературы по компьютерному зрению с инженерными подходами к построению распределенных систем и нормативной базой по оформлению научно-технической документации формирует методический контур настоящей работы.

1.2.2 Обзор существующих платформ видеоаналитики

Среди представленных на рынке платформ интеллектуального видеонаблюдения можно условно выделить три основные группы. К первой относятся системы класса VMS (Video Management System) с расширенными модулями видеоаналитики, например *Macroscop*, *TRASSIR* и *Axxon One*; открытые материалы производителей описывают их как платформы видеонаблюдения с развитыми аналитическими модулями [33—35]. Ко второй группе относятся облачные сервисы видеонаблюдения с аналитикой, например *Ivideon*; они снижают порог входа для пользователя, однако предполагают передачу видеоданных во внешний контур обработки [36]. Третью группу образуют международные платформы с акцентом на машинное обучение и расширяемую аналитику, например *Milestone XProtect* и *Vaidio*; такие решения обладают развитой функциональностью, но их открытые материалы не раскрывают внутренний алгоритмический и интеграционный контур в объеме, достаточном для воспроизводимого исследовательского стенда [37; 38].

Сравнение характеристик представленных платформ и разрабатываемого решения по существенным для исследования критериям приведено в таблице 1.1. Сравнение выполнено по открытым материалам производителей; если сведения о внутреннем устройстве продукта в них не раскрыты, это фиксируется как ограничение открытой проверяемости, а не как утверждение о фактическом отсутствии соответствующего механизма.

Таблица 1.1 - Сравнение существующих платформ видеоаналитики и разрабатываемого решения

Критерий сравнения	Macroscop	TRASSIR	Ivideon	Vaidio	Платформа
Контур применения по открытым материалам	локальный/РФ	локальный/РФ	облачный сервис	международный	локальный стенд/РФ
Тип развертывания	локальное	локальное	облачное	локальное / облачное	локальное
Публично раскрытое алгоритмическое ядро	не раскрыто	не раскрыто	не раскрыто	не раскрыто	открытое YOLO-ядро
Возможность замены детектора пользователем	ограничена	ограничена	нет	ограничена	да (по контракту)
Многообъектное сопровождение с признаками внешнего вида	частично	да	частично	да	да, DeepSORT-подход
Правиловой движок событий (zone_enter, line_cross, dwell_time)	да	да	частично	да	да
Надежная асинхронная доставка событий (исходящие сообщения + очередь недоставленных сообщений)	не раскрыта	не раскрыта	облачный контур	не раскрыта	да, на открытом стеке
Открытые контракты API/событий	ограничены	ограничены	ограничены	ограничены	да, OpenAPI
Прозрачные операционные метрики (Prometheus / Grafana)	не заявлены как открытый контур	не заявлены как открытый контур	нет	не заявлены как открытый контур	да
Воспроизводимость стенда (compose/IaC)	не заявлена	не заявлена	не заявлена	не заявлена	да
Стоимость владения	лицензия + поддержка	лицензия + поддержка	подписка	лицензия + поддержка	открытый стенд

Анализ показывает, что существующие платформы успешно решают значительную часть прикладных задач охранного видеонаблюдения, однако обладают рядом ограничений, существенных для научно-исследовательской работы и построения предметно-ориентированных решений:

- ограниченная раскрытость алгоритмического ядра в открытых материалах затрудняет адаптацию детектора и модуля сопровождения к специфическим условиям конкретного объекта (например, обнаружение БПЛА на дальних дистанциях);
- ограниченная открытость контрактов событий и программных интерфейсов (API) усложняет интеграцию с произвольными внешними системами и сравнительные исследования;
- ограниченная прозрачность операционных метрик затрудняет независимую оценку эксплуатационных характеристик (сквозной задержки, устойчивости доставки, влияния отказов);

- модель развертывания платформ преимущественно ориентирована на эксплуатационное внедрение, а не на воспроизводимый исследовательский стенд.

Выявленные ограничения формируют практическую и научную нишу для разрабатываемого решения: открытый алгоритмический и интеграционный контур, фиксированные контракты событий, прозрачные метрики и воспроизводимый стенд развертывания позволяют проводить экспериментальную оценку архитектурных и алгоритмических решений в условиях, сопоставимых с реальными системами охраны, и при этом сохранять контроль за исходным кодом, конфигурацией и наблюдаемостью.

1.2.3 Выводы по обзору

Результаты обзора литературы и существующих платформ позволяют сделать следующие основные выводы:

- 1) для задач охранной видеоаналитики реального времени накоплена развитая алгоритмическая база (YOLO-семейство, SORT/DeepSORT, метрики MOT-Challenge), пригодная для построения базового конвейера обработки;
- 2) инженерные шаблоны микросервисной архитектуры (транзакционная таблица исходящих сообщений, контролируемые повторы, очереди недоставленных сообщений) обеспечивают необходимый уровень надежности доставки событий и могут быть перенесены в предметную область охранного видеонаблюдения;
- 3) существующие промышленные платформы решают значительную часть задач, однако их закрытость и ограниченная прозрачность контрактов и метрик создают научно-методическую нишу для построения открытого исследовательского решения с фиксированными контрактами и воспроизводимым стендом.

1.3 Требования к разрабатываемой платформе

Формирование требований необходимо для корректного выбора методов и архитектурных решений. Поскольку разрабатываемая система ориентирована на реальную эксплуатацию в составе охранного видеонаблюдения

ния, требования должны охватывать как функциональные возможности, так и ограничения по качеству обслуживания.

1.3.1 Функциональные требования

К основным функциональным требованиям относятся:

- 1) прием одного или нескольких видеопотоков от камер наблюдения и поддержка их непрерывной обработки;
- 2) выделение целевых объектов в кадре с определением их пространственного положения и класса;
- 3) сопровождение обнаруженных объектов во времени с присвоением устойчивых идентификаторов;
- 4) поддержка анализа нескольких классов объектов, включая людей, транспортные средства и БПЛА;
- 5) формирование событий на основе пространственно-временных правил, связанных с зонами контроля, виртуальными линиями и логикой поведения объекта;
- 6) сохранение результатов обработки в форме метаданных, журналов событий и параметров траекторий;
- 7) передача сведений о событиях внешним потребителям через программные интерфейсы или каналы интеграции;
- 8) возможность настройки состава правил, перечня контролируемых зон и параметров обработки без изменения базовой логики платформы.

1.3.2 Нефункциональные требования

Для рассматриваемого класса систем определяющее значение имеют следующие нефункциональные требования:

- *Работа в реальном времени.* Платформа должна обеспечивать обработку видеопотоков с ограниченной задержкой, достаточной для практического реагирования на инцидент.
- *Масштабируемость.* Архитектура должна допускать увеличение числа камер, правил событийного анализа и внешних потребителей без полной переработки системы.
- *Отказоустойчивость.* Отдельные сбои обработки, кратковременная

недоступность камер или каналов связи не должны приводить к потере работоспособности всей платформы.

- *Точность и устойчивость.* Методы должны сохранять приемлемое качество работы при частичных перекрытиях объектов, изменениях освещенности и наличии фоновых помех.
- *Расширяемость.* Система должна допускать замену или дообучение детектора, замену модуля сопровождения и уточнение набора событийных правил без кардинальной перестройки конвейера.
- *Интегрируемость.* Результаты работы платформы должны представляться в форме, совместимой с внешними системами хранения, мониторинга и оповещения.
- *Интерпретируемость.* Для задач безопасности важно не только обнаружить событие, но и объяснить причину его формирования через траекторию объекта, тип правила и контекст наблюдения.

Совокупность перечисленных требований определяет необходимость выбора методов, сочетающих высокую скорость обработки с устойчивой работой на реальных видеоданных и возможностью последующей интеграции в событийно-ориентированную программную платформу.

1.4 Обоснование выбора методов

1.4.1 Выбор метода обнаружения объектов

Методы обнаружения объектов, применимые в системах видеонаблюдения, можно условно разделить на несколько групп.

Классические методы, основанные на вычитании фона, межкадровой разности, пороговой обработке и выделении контуров, отличаются сравнительно низкой вычислительной сложностью и могут эффективно использоваться для выявления факта движения при статичной камере. Однако данные методы плохо переносят сложные погодные условия, переменное освещение, тени, дрожание камеры и не обеспечивают устойчивой классификации объектов по типам. Для задач событийного анализа этого недостаточно, поскольку системе необходимо различать классы объектов и корректно обрабатывать сцены с несколькими целями.

Двухэтапные нейросетевые методы обнаружения, например подходы

семейства R-CNN, обеспечивают высокую точность локализации и классификации объектов. Их недостатком является относительно высокая вычислительная стоимость, особенно при обработке многоканальных видеопотоков в режиме реального времени. В рассматриваемой задаче чрезмерная задержка обработки снижает практическую ценность решения, даже если точность отдельных детекций остается высокой.

Методы трансформерного типа представляют значительный научный интерес и демонстрируют хорошие результаты в ряде задач компьютерного зрения. Вместе с тем их эффективное применение обычно требует существенных вычислительных ресурсов, значительного объема данных для адаптации и более сложного контура развертывания. Для платформы, ориентированной на эксплуатационные сценарии охранного видеонаблюдения, приоритетным является более предсказуемый баланс между скоростью, точностью и простотой практического внедрения.

С учетом поставленных требований наиболее целесообразным является использование одноэтапного детектора семейства YOLO [1; 27; 28]. Выбор данного класса методов обусловлен следующими факторами:

- обнаружение и классификация выполняются в едином вычислительном конвейере, что обеспечивает высокую скорость обработки;
- методы семейства YOLO хорошо адаптированы к потоковой обработке видео и практическому развертыванию на GPU- и edge-платформах;
- использование многомасштабного представления признаков позволяет обнаруживать объекты различных размеров, что существенно для задач наблюдения за протяженными сценами и для сценариев обнаружения БПЛА;
- архитектуры данного класса допускают дообучение на предметно-ориентированных наборах данных, что позволяет адаптировать модель к особенностям конкретных охраняемых зон.

Следовательно, в качестве базового метода обнаружения в работе целесообразно использовать одноэтапный детектор семейства YOLO, обеспечивающий приемлемое сочетание быстродействия и качества распознавания для задач реального времени.

1.4.2 Выбор метода сопровождения объектов

После обнаружения объектов необходимо обеспечить их сопровождение между кадрами. Изолированная покадровая детекция не позволяет надежно анализировать поведение объекта, определять направление движения и выделять события, зависящие от длительности нахождения в зоне или пересечения рубежей.

Методы, основанные только на оптическом потоке или корреляционных фильтрах, подходят преимущественно для сопровождения отдельных объектов и подвержены накоплению ошибки при длительном наблюдении. Алгоритм SORT обладает высокой скоростью и хорошо подходит для сценариев с качественными детекциями и небольшим числом перекрытий, однако использует в основном кинематические признаки и потому чувствителен к переключениям идентификаторов в сложных сценах.

Для задач охранного видеонаблюдения более обоснованным является использование подхода сопровождения по результатам обнаружения, сочетающего модель движения объекта и процедуру ассоциации новых детекций с уже существующими траекториями. Среди практических методов этого класса целесообразно выделить DeepSORT, который дополняет кинематическое сопоставление признаками внешнего вида объекта [2; 26]. Это особенно важно при частичных перекрытиях, кратковременных пропаданиях детекций и наличии нескольких близко расположенных целей.

Выбор метода сопровождения типа DeepSORT обусловлен следующими причинами:

- использование фильтра Калмана позволяет прогнозировать положение объекта и сглаживать влияние пропусков детекций;
- задача ассоциации решается формализованно и допускает устойчивую работу в многообъектных сценах;
- применение дескрипторов внешнего вида снижает число переключений идентификаторов по сравнению с чисто кинематическими методами сопровождения;
- выход модуля сопровождения естественным образом формирует траектории, необходимые для последующего событийного анализа.

Таким образом, для подсистемы сопровождения обоснован выбор мно-

гообъектного модуля сопровождения по результатам обнаружения, реализуемого на основе фильтра Калмана, процедуры ассоциации и признаков внешнего вида, что соответствует методологической логике DeepSORT.

1.4.3 Выбор метода событийного анализа

Событийный анализ в системах охранного видеонаблюдения может быть реализован различными способами. Один из подходов предполагает прямое распознавание сложных событий по видеоклипам с помощью специализированных нейросетевых моделей. Такой подход перспективен, однако требует больших объемов размеченных данных именно по событиям, менее прозрачен при интерпретации результата и хуже адаптируется к локальным регламентам конкретного объекта.

Для практико-ориентированной платформы более целесообразным является событийный анализ на основе правил, применяемых к траекториям объектов и конфигурации контролируемых зон. В этом случае событие трактуется как выполнение некоторого пространственно-временного предиката: пересечение виртуальной линии, вход в запретную область, нахождение в зоне дольше порогового интервала, появление объекта заданного класса в определенном секторе наблюдения и т.п. Такой подход лучше согласуется с архитектурным шаблоном надежной асинхронной интеграции через транзакционную таблицу исходящих сообщений [3; 13].

Выбор правилowego подхода обусловлен следующими преимуществами:

- интерпретируемость результата и возможность явного объяснения причины формирования события;
- простота адаптации к конкретным требованиям эксплуатации без полного переобучения модели;
- независимость логики событийного анализа от конкретной реализации детектора или модуля сопровождения;
- удобство интеграции с подсистемами оповещения, журналирования и операторского контроля.

Следовательно, в рамках разрабатываемой платформы рационально использовать композицию из трех уровней: детектор семейства YOLO для по-

кадрового обнаружения, модуль сопровождения на основе DeepSORT и модуль правилowego событийного анализа для формирования целевых событий.

1.5 Сравнительный анализ альтернатив реализации платформы

Помимо выбора алгоритмов компьютерного зрения, для практической пригодности системы требуется выбрать технологический стек контура исполнения: способ захвата потоков, модель обмена сообщениями и архитектуру хранения результатов. Ниже приведено сопоставление основных альтернатив.

1.5.1 Альтернативы по подсистеме захвата видеопотоков

Таблица 1.2 - Сравнение альтернатив захвата видеопотоков

Альтернатива	Оценка применимости
OpenCV VideoCapture	Быстрый старт для системы, но ограниченный контроль над буфером, reconnect-политикой и диагностикой транспортного уровня
GStreamer	Высокая гибкость, но более сложная конфигурация и порог вхождения на начальном этапе внедрения
FFmpeg/PyAV	Компромисс между управляемостью потока, зрелостью экосистемы и интеграцией с Python-контуром

Для текущей работы выбран FFmpeg/PyAV, поскольку это решение дает предсказуемое поведение при сбоях RTSP и не требует усложнения стека на раннем этапе [9; 10].

1.5.2 Альтернативы по интеграционному контуру

Таблица 1.3 - Сравнение вариантов интеграции событий

Вариант	Ограничения/преимущества
Синхронные HTTP-вызовы из горячего пути	Простая реализация, но высокий риск роста задержки и каскадной деградации при недоступности получателя
Kafka-контур на старте	Сильные возможности масштабирования, но избыточная инфраструктурная сложность для диапазона 1--10 камер
RabbitMQ + таблица исходящих сообщений	Достаточная надежность доставки, подтверждения публикации, очередь недоставленных сообщений и умеренный порог сопровождения на односерверном стенде

С учетом требований отказоустойчивости и ограниченного масштаба пилотного стенда выбран вариант RabbitMQ + транзакционная таблица исходящих сообщений [3; 24; 31].

1.5.3 Альтернативы по хранилищу

Таблица 1.4 - Сравнение подходов к хранению результатов

Вариант	Комментарий
NoSQL-only	Гибкость схемы, но менее удобная транзакционная связка для <code>events</code> + <code>event_outbox</code>
PostgreSQL + BLOB-видео	Упрощает единое хранилище, но приводит к раздуванию БД и росту стоимости операций
PostgreSQL + файловые артефакты	Транзакционная надежность структурированных данных при контролируемом объеме БД и выносе тяжелых артефактов на файловый слой

В работе выбирается третий вариант: события и конфигурация хранятся в PostgreSQL, а снимки/клипы - в файловом хранилище с сохранением ссылок и метаданных в БД [32].

1.5.4 Архитектурные компромиссы

Принятый стек формирует набор осознанных компромиссов:

- в пользу управляемости: отказ от избыточно сложного распределенного контура на текущем этапе;
- в пользу интерпретируемости: правилый анализ вместо «черного ящика» событийной нейросети;
- в пользу надежности: транзакционная таблица исходящих сообщений, повторные попытки и очередь недоставленных сообщений вместо публикации без гарантий доставки;
- в пользу воспроизводимости: единый compose-стенд и фиксированные контракты API/сообщений.

Такие компромиссы соответствуют логике магистерского исследования: сначала подтвердить корректность целевого контура и измеримые свойства интеграционной эксплуатации на стенде, затем масштабировать отдельные подсистемы.

1.6 Исходные данные для разработки и экспериментальной оценки

Исходные данные для построения и оценки платформы должны отражать реальные условия эксплуатации систем охранного видеонаблюдения и обеспечивать возможность проверки как алгоритмических, так и архитектурных решений.

1.6.1 Состав входных данных

К основным видам исходных данных относятся:

- 1) *Видеопотоки с камер наблюдения.* Данные потоки являются основным источником информации и должны включать сцены с различными условиями освещения, погодными факторами, уровнями плотности объектов, масштабами наблюдения и степенью подвижности сцены.
- 2) *Размеченные изображения и видеопоследовательности.* Для обучения

и валидации детектора требуются данные с разметкой ограничивающих рамок и классов объектов. Для оценки сопровождения дополнительно необходимы идентификаторы траекторий или эквивалентная разметка перемещения объектов во времени.

- 3) *Конфигурация контролируемых зон.* К этой группе относятся полигоны запретных областей, виртуальные линии пересечения, описания периметров, приоритеты сценариев и параметры правил событийного анализа.
- 4) *Метаданные камер и потоков.* Необходимы идентификаторы камер, временные метки, характеристики потока, а также сведения о пространственном контексте установки камеры.
- 5) *Журналы и эталонные события.* Для оценки модуля событийного анализа требуется набор сценариев или журналов, позволяющих сопоставить сформированные платформой события с ожидаемыми результатами.

Содержательно входные данные должны покрывать не только типовые сценарии, но и сложные случаи, наиболее критичные для задач безопасности: частичные перекрытия объектов, изменение фона, появление мелких объектов на большом расстоянии, резкие изменения освещения, ложные движения, вызванные погодными факторами, а также сцены с несколькими целями одновременно.

1.6.2 Требования к подготовке данных

Для корректной разработки и экспериментальной оценки платформы исходные данные должны удовлетворять ряду требований:

- классовый состав разметки должен соответствовать целевым объектам наблюдения;
- данные должны включать различные ракурсы, масштабы и режимы съемки;
- обучающая и тестовая выборки должны быть разделены таким образом, чтобы оценка качества отражала способность методов к обобщению;
- при наличии сценариев с БПЛА требуется отдельное покрытие малыми

объектами и сценами дальнего наблюдения;

- временная разметка событий должна быть синхронизирована с видеопотоками и траекториями объектов.

Наличие структурированных исходных данных особенно важно для раздельной оценки отдельных подсистем платформы: детектора, модуля сопровождения, модуля событийного анализа и механизма передачи сведений о событиях.

1.6.3 Показатели оценки качества

Исходные данные должны обеспечивать возможность вычисления показателей, связанных с целью работы и последующими этапами апробации. К таким показателям относятся:

- точность обнаружения объектов, оцениваемая через метрики качества детекции;
- устойчивость сопровождения, характеризуемая непрерывностью траекторий, числом переключений идентификаторов и качеством ассоциации;
- корректность генерации событий, включая долю пропущенных и ложных событий;
- задержка обработки видеопотока и суммарное время от появления объекта до формирования события;
- надежность доставки сведений о событиях внешним потребителям.

Фиксация указанных показателей на этапе постановки задачи необходима, поскольку именно они позволяют связать выбор методов с целями исследования и впоследствии использовать единый набор критериев при экспериментальной проверке разработанной платформы. При этом показатели `r95 object->event`, доля отброшенных кадров и метрики событийной точности задаются как целевые показатели расширенной методики; в текущей апробации количественно проверяются интеграционная доставка, отказные сценарии, RBAC-ограничения и демонстрационный БПЛА-контур.

1.7 Связь требований с измеряемыми метриками и единой нотацией

Для обеспечения сквозной сопоставимости главы 1 с главами 2--4 вводится единая нотация входов и выходов конвейера:

$$\mathcal{V} \rightarrow \mathcal{D} \rightarrow \mathcal{T} \rightarrow \mathcal{E} \rightarrow \text{event_outbox}. \quad (5)$$

где $\mathcal{V}$ — видеопотоки;

$\mathcal{D}$ — детекции;

$\mathcal{T}$ — треки;

$\mathcal{E}$ — сформированные события.

Такое обозначение далее используется при математической формализации (глава 2), инженерной реализации (глава 3) и в таблицах апробации (глава 4).

Требования главы 1 связываются с метриками следующим образом:

Таблица 1.5 - Трассировка требований и измеряемых метрик

Требование	Ключевые метрики проверки
Работа в реальном времени	p95 object->event, длина очередей, доля отброшенных кадров
Устойчивость сопровождения	переключения идентификаторов (<i>ID-switches</i>), доля прерванных траекторий
Корректность событий	точность и полнота событий, число ложных/пропущенных событий
Надежная доставка	размер очереди исходящих сообщений, число повторных попыток, число сообщений в очереди недоставленных сообщений, доля успешно доставленных событий
Безопасность доступа	успешность сценария JWT-аутентификации, матрица RBAC-ограничений

Таким образом, уже на этапе постановки задачи формируется измеримая основа для верификации результатов разработки, что соответствует требованиям воспроизводимости инженерных экспериментов по ГОСТ 7.32 [17]

и практикам эксплуатационного мониторинга [7; 15].

Выводы по главе 1

В первой главе выполнен анализ предметной области и сформулирована постановка задачи разработки платформы видеоаналитики реального времени для систем охранного видеонаблюдения. Основные результаты главы могут быть резюмированы следующими положениями.

Во-первых, на основе анализа особенностей предметной области показано, что решение рассматриваемой задачи требует согласования четырех уровней обработки: приема видеопотоков, обнаружения объектов в кадре, их сопровождения во времени и событийного анализа траекторий. Сформированные пользовательские роли (`operator`, `admin`, `viewer`) и карты пути пользователя позволили согласовать функциональные требования с реальной деятельностью операторов и администраторов охраняемых объектов.

Во-вторых, проведенный обзор существующих платформ видеоаналитики и систематизация литературы по компьютерному зрению, многообъектному сопровождению и микросервисным архитектурам позволили выявить научно-методическую нишу настоящей работы: построение открытого исследовательского решения с фиксированными контрактами событий, прозрачными операционными метриками и воспроизводимым стендом развертывания. Сравнение по таблице 1.1 показывает, что существующие промышленные платформы хорошо закрывают типовые сценарии охранной видеоаналитики, однако ограничивают возможности сравнительных исследований и адаптации алгоритмического ядра.

В-третьих, на основе функциональных и нефункциональных требований обоснован выбор одноэтапного детектора семейства YOLO, многообъектного модуля сопровождения на основе DeepSORT и правилowego подхода к формированию событий, а также выбор технологического стека контура исполнения (FFmpeg/PyAV, FastAPI, PostgreSQL, RabbitMQ, Prometheus, Grafana). Определены состав исходных данных, требования к их подготовке и ключевые показатели качества (`p95 object` → `event`, переключения идентификаторов, точность и полнота событий, размер очереди исходящих сообщений, число повторов и сообщений в очереди недоставленных), которые далее используются как методическая основа; в главе 4 явно разграни-

чиваются фактически измеренные показатели и показатели, вынесенные в расширенные прогоны.

Совокупность сформированных требований, обоснованного выбора методов и измеримых показателей качества образует техническое задание на разработку платформы и формирует методическую основу для дальнейшего проектирования архитектуры, программной реализации и экспериментальной оценки результатов.

2 ТЕОРЕТИЧЕСКИЕ ПРЕДПОСЫЛКИ, МОДЕЛИ И МЕТОДОЛОГИЯ

2.1 Формальная модель конвейера обработки

Сквозной маршрут платформы графически представлен на рисунке 2.1: от приема RTSP-потока до фиксации события в БД и его асинхронной доставки во внешний контур. Эта схема используется как опорная при формализации модели и при описании реализации в главе 3.

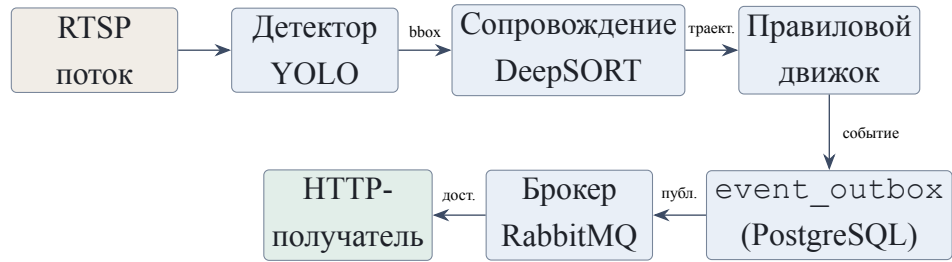


Рисунок 2.1 - Сквозной конвейер обработки видеопотока и доставки событий

Платформа рассматривается как композиция взаимосвязанных функций

$$\Phi = \Phi_{\text{outbox}} \circ \Phi_{\text{rules}} \circ \Phi_{\text{track}} \circ \Phi_{\text{detect}} \circ \Phi_{\text{capture}}, \quad (1)$$

где $i \in \{1, \dots, N\}$ — индекс камеры;

Φ — сквозной конвейер обработки видеопотока для камеры i .

[10; 22; 28]

Для камеры i в момент времени t на вход поступает поток кадров x_t^i .

Стадия захвата и нормализации формирует последовательность

$$\hat{x}_t^i = \Phi_{\text{capture}}(x_t^i, m_i), \quad (2)$$

где m_i — метаданные источника: временная зона, целевой FPS и стратегия переключения.

На критическом пути применяется политика замещения последним актуальным кадром: очередь между захватом и инференсом ограничена разме-

ром 1. Формально буфер $\mathcal{Q}_i$ в момент t определяется как

$$\mathcal{Q}_i(t) = \begin{cases} \{\hat{x}_t^i\}, & \text{если буфер пуст;} \\ \{\hat{x}_t^i\}, & \text{если буфер содержит устаревший кадр.} \end{cases} \quad (3)$$

Такой режим эквивалентен оператору замещения последнего элемента и ограничивает накопление задержки [9].

Детектор формирует множество кандидатов

$$\mathcal{D}_t^i = \Phi_{\text{detect}}(\hat{x}_t^i; \theta_d), \quad (4)$$

где θ_d — параметры нейросетевой модели класса YOLO.

[1; 27; 28]

Модуль сопровождения преобразует детекции в траектории:

$$\mathcal{T}_t^i = \Phi_{\text{track}}(\mathcal{D}_t^i, \mathcal{T}_{t-1}^i; \theta_t). \quad (5)$$

Результатом являются активные треки с устойчивыми `track_id` и обновленными состояниями [2; 26].

Правиловой движок вычисляет предикаты событий:

$$\mathcal{E}_t^i = \Phi_{\text{rules}}(\mathcal{T}_t^i, \mathcal{Z}_i, \mathcal{R}_i), \quad (6)$$

где $\mathcal{Z}_i$ — геометрия зон и линий контроля камеры i ;

$\mathcal{R}_i$ — конфигурация правил событийного анализа камеры i .

Наконец, выход конвейера фиксируется транзакционно:

$$(\text{events}, \text{event_outbox}) = \Phi_{\text{outbox}}(\mathcal{E}_t^i). \quad (7)$$

Тем самым обеспечивается логическая атомарность между фактом обнаруженного события и фактом его последующей публикации во внешний брокер [3; 24].

2.2 Обоснование архитектуры и технологического стека

В данном разделе формализуется выбор архитектурного решения и технологического стека платформы. Согласно методическим рекомендациям, во

второй главе ВКР должна быть не только приведена архитектура и используемый набор технологий, но и обосновано, почему выбраны именно они. Обоснование строится на трех уровнях: архитектурный паттерн, алгоритмическое ядро и инфраструктурный стек.

2.2.1 Архитектурный паттерн

Платформа реализована в виде набора слабо связанных микросервисов, взаимодействующих через явно описанные программные контракты. Сравнение основных архитектурных альтернатив для рассматриваемого класса задач приведено в таблице 2.1.

Таблица 2.1 - Сравнение архитектурных паттернов

Паттерн	Оценка применимости
Монолитное приложение	Простое развертывание, но смешение алгоритмического и интеграционного контуров повышает риск каскадных отказов и затрудняет независимое масштабирование
Модульный монолит	Снижает связность по сравнению с обычным монолитом, однако сохраняет общую модель отказов и общий контур исполнения
Микросервисы	Изолируют контуры обработки видео, конфигурации и интеграции; обеспечивают независимое масштабирование, но требуют надежного транспорта сообщений и наблюдаемости
Serverless / FaaS	Пригодны для эпизодических задач, но плохо согласуются с непрерывной потоковой обработкой видеоданных

Для платформы видеоаналитики реального времени выбран микросервисный архитектурный шаблон с разделением на контур управления (управление, конфигурация, доступ) и контур обработки данных (обработка видео, генерация и доставка событий). Такое разделение соответствует рекомендациям инженерной литературы по построению событийно-ориентированных

систем [3; 13] и обеспечивает выполнение архитектурных факторов, сформулированных в главе 1: отказ интеграционного контура не должен блокировать аналитическое ядро, а замена детектора или модуля сопровождения должна быть локальным изменением.

2.2.2 Алгоритмическое ядро: детекция и сопровождение

Выбор алгоритмов обнаружения и сопровождения объектов рассмотрен в разделе 1.2 главы 1. Здесь формализуем критерии и итоговое решение в табличной форме (таблица 2.2).

Таблица 2.2 - Обоснование выбора алгоритмов обнаружения и сопровождения

Подсистема	Выбранный метод	Ключевые причины выбора
Детектор объектов	YOLO-семейство (одноэтапный) [1; 27; 28]	Высокая скорость инференса, многомасштабное представление признаков, доступность открытых реализаций, возможность дообучения на предметных данных, в том числе для малых объектов (БПЛА)
Сопровождение объектов	DeepSORT-подход на основе фильтра Калмана и признаков внешнего вида [2; 8; 29]	Сочетание кинематического прогноза и признаков внешнего вида снижает число переключений идентификаторов (ID-switches) при перекрытиях; формальная процедура ассоциации совместима с гейтингом и многокамерной изоляцией
Событийный анализ	Правилковой движок над траекториями [3]	Интерпретируемость событий, простота адаптации к локальным регламентам, независимость от конкретной реализации детектора и модуля сопровождения

Альтернативные подходы (двухэтапные и трансформерные детекторы,

end-to-end событийные нейросети) обладают преимуществами по точности на сложных сценах, однако предъявляют существенно более высокие требования к вычислительным ресурсам и объему размеченных данных по событиям, что не согласуется с целями практико-ориентированного исследовательского стенда.

2.2.3 Веб-фреймворк управляющего сервиса

Сервис `api` реализует управляющий контур: аутентификацию, авторизацию, управление конфигурацией камер, зон, правил и доступом к журналу событий. Сравнение основных альтернатив для Python-экосистемы приведено в таблице 2.3.

Таблица 2.3 - Сравнение веб-фреймворков для управляющего сервиса

Фреймворк	Оценка применимости
Django	Полнофункциональный фреймворк с ORM и admin-панелью, но избыточен для тонкого API-контура и ограничен по асинхронной модели исполнения
Flask	Минималистичный фреймворк, но требует ручной реализации валидации, асинхронности и автодокументации API
FastAPI [22]	Современный асинхронный фреймворк, основанный на Starlette и Pydantic; обеспечивает автоматическую генерацию OpenAPI-схемы, строгую валидацию входных данных и хорошую совместимость с асинхронным стеком

Для управляющего сервиса выбран FastAPI [22] в сочетании с SQLAlchemy и Alembic [23; 39]. Основные мотивы: автоматическая генерация спецификации OpenAPI и контроль контракта на этапе разработки; нативная асинхронная модель исполнения, согласованная с типичной нагрузкой управляющего API; типобезопасные модели данных на основе Pydantic, что снижает риск рассинхронизации схемы и кода.

2.2.4 Система управления базами данных

Хранилище платформы обслуживает структурированные сущности (пользователи, роли, камеры, зоны, правила, события) и таблицу `event_outbox`, требующую транзакционной согласованности с записью событий. Сравнение СУБД приведено в таблице 2.4.

Таблица 2.4 - Сравнение систем управления базами данных

СУБД	Оценка применимости
SQLite	Подходит для прототипов, но не предназначена для конкурентной нагрузки и работы под несколькими сервисами
MySQL/ MariaDB	Зрелая реляционная СУБД, однако ряд особенностей реализации (например, поведение блокировок, поддержка JSON и расширений) уступает PostgreSQL
PostgreSQL	Реляционная СУБД с богатой поддержкой расширений, типов JSON/JSONB и продвинутых индексов; обеспечивает надежную транзакционную семантику, необходимую для шаблона транзакционной таблицы исходящих сообщений
MongoDB	Документная СУБД, удобная для гибких схем, но менее естественна для строгой транзакционной фиксации <code>events</code> + <code>event_outbox</code> в одной операции

Выбор PostgreSQL [32] обусловлен необходимостью атомарной фиксации записей в `events` и `event_outbox` в рамках одной транзакции, а также удобством работы с полем полезной нагрузки `payload` в формате JSONB и наложением уникального ограничения на поле `dedup_key`. Эти возможности непосредственно используются в формальной модели надежной доставки.

2.2.5 Брокер сообщений

Интеграционный контур платформы строится на основе шаблона транзакционной таблицы исходящих сообщений с асинхронной публикацией доменных событий во внешние системы. В таблице 2.5 приведено сопоставле-

ние основных вариантов.

Таблица 2.5 - Сравнение брокеров сообщений

Брокер	Оценка применимости
Apache Kafka	Высокая пропускная способность и упорядоченные журналы, но избыточная инфраструктурная сложность для пилотного стенда с числом камер 1--10
Redis Streams	Простота развертывания, но менее зрелые гарантии надежной доставки и менее богатая модель маршрутизации
RabbitMQ [14; 24; 31]	Зрелая модель маршрутизации через обменники, очереди и привязки, подтверждения публикации RabbitMQ и обменник недоставленных сообщений

Выбор RabbitMQ обусловлен сочетанием подтвержденной публикации, штатной поддержки очередей недоставленных сообщений и достаточной для пилотного стенда пропускной способности [14; 24; 31]. Это позволяет реализовать формализованную в дальнейшем модель повторных попыток и приближенную оценку вероятности успешной доставки без введения избыточно сложной инфраструктуры.

2.2.6 Подсистема видеозахвата

Для приема RTSP-потоков и их декодирования сопоставлены три варианта: OpenCV VideoCapture, GStreamer и FFmpeg/PyAV (см. таблицу 1.2 в главе 1). В платформе использован FFmpeg/PyAV [9; 10] как компромисс между управляемостью потока, зрелостью экосистемы и интеграцией с Python-контуром исполнения. Это решение позволяет реализовать политику замещения кадра последним актуальным кадром, формализованную в разделе 2.1.

2.2.7 Наблюдаемость и оркестрация

Для воспроизводимого развертывания и эксплуатационной приемки выбраны Docker Compose [25], Prometheus [7] и Grafana [15]. Docker Compose

обеспечивает декларативное описание стенда и согласованность окружений между разработкой и приемочными прогонами; Prometheus реализует сбор и хранение временных рядов, Grafana - их визуализацию и настройку предупреждений. Альтернативы (Kubernetes, Datadog, ELK-стек) обладают большей мощностью, но для пилотного односерверного стенда вносят избыточную сложность сопровождения, не оправданную целями исследования.

2.2.8 Сводный технологический стек

Итоговый технологический стек платформы зафиксирован в таблице 2.6. Для каждой подсистемы указаны выбранное решение и ключевые источники, обосновывающие этот выбор.

Таблица 2.6 - Сводный технологический стек платформы

Подсистема	Выбранное решение	Назначение и обоснование
Управляющий API	FastAPI + SQLAlchemy + Alembic [22; 23; 39]	Тонкий асинхронный API с автогенерацией OpenAPI-схемы и строгой валидацией данных
Хранилище данных	PostgreSQL [32]	Транзакционная фиксация events + event_outbox, поддержка JSONB, уникальные индексы для дедупликации
Брокер сообщений	RabbitMQ [14; 24; 31]	Подтвержденная публикация, маршрутизация по типам событий, штатная очередь недоставленных сообщений
Видеозахват	FFmpeg и PyAV [9; 10]	Управляемый декодер RTSP с предсказуемым поведением при сбоях
Детектор	YOLO-семейство [1; 27; 28]	Одноэтапная архитектура с приемлемой задержкой инференса
Модуль сопровождения	DeepSORT-подход [2; 8; 29]	Сочетание кинематического прогноза и признаков внешнего вида
Метрики и мониторинг	Prometheus + Grafana [7; 15]	Эксплуатационная наблюдаемость и регламент приемки
Оркестрация стенда	Docker Compose [25]	Декларативный воспроизводимый стенд для разработки и приемки

Сводный стек согласован между собой: все компоненты имеют открытые реализации, поддерживают типовые контракты обмена данными (HTTP, AMQP, SQL) и обеспечивают совместное выполнение требований к надежности доставки, наблюдаемости и воспроизводимости стенда. Тем самым обеспечивается прямая связь между сформулированными в главе 1 требованиями и инженерным контуром, формализуемым ниже.

Композиция сервисов и их связи показаны на рисунке 2.2 в нотации C4 (контейнерный уровень). Слева расположены внешние участники взаимодействия (камеры, оператор), в центре - управляющий контур (api, ui, PostgreSQL), справа - контур обработки событий (analytics-worker, RabbitMQ, integration-worker) и внешние получатели событий. Подсистема наблюдаемости (Prometheus и Grafana) собирает метрики со всех сервисов через интерфейсы /metrics.

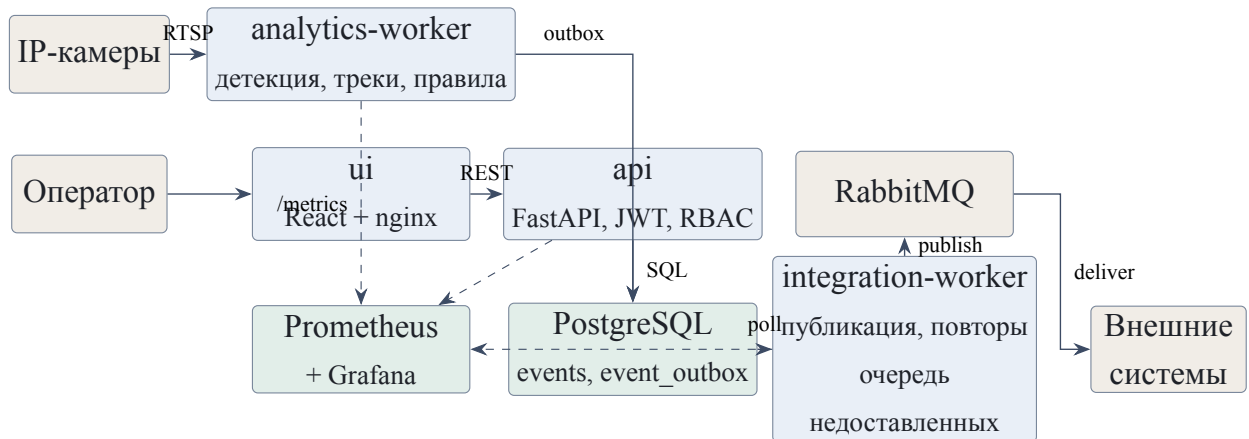


Рисунок 2.2 - Архитектура платформы (C4, контейнерный уровень)

Диаграмма последовательности надежной асинхронной доставки события представлена на рисунке 2.3. Транзакционная фиксация events + event_outbox в одной операции БД обеспечивает атомарность факта возникновения события; последующая публикация в RabbitMQ и доставка HTTP-получателю выполняются асинхронно с возможностью повторов и перевода в очередь недоставленных сообщений при исчерпании попыток.

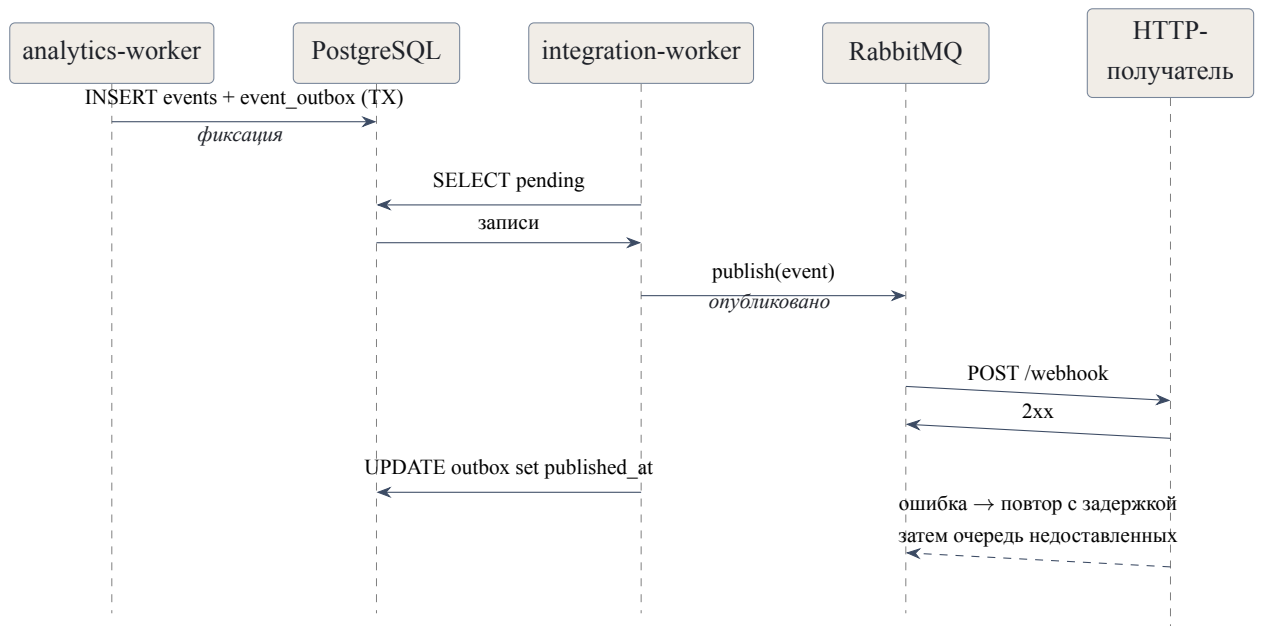


Рисунок 2.3 - Диаграмма последовательности надежной доставки события через транзакционную таблицу исходящих сообщений

2.2.9 Логическая модель работы платформы

Логика работы платформы укрупненно описывается следующей последовательностью шагов:

- 1) *Управляющий контур.* Администратор системы через API регистрирует камеры, описывает геометрию зон и линий, задает набор правил событийного анализа и распределяет роли пользователей;
- 2) *Активация камеры.* Сервис `analytics-worker` получает конфигурацию из API и инициирует прием RTSP-потока, политику декодирования и работу детектора;
- 3) *Основной цикл обработки.* Для каждого актуального кадра выполняется последовательность «детекция → фильтрация классов → обновление модуля сопровождения → применение правил»;
- 4) *Транзакционная фиксация.* При выполнении предиката правила в одной транзакции записываются строки в `events` и `event_outbox`, что исключает потерю события на границе БД;
- 5) *Асинхронная публикация.* Сервис `integration-worker` опрашивает `event_outbox`, публикует сообщения в RabbitMQ через подтвержденный канал и обновляет статус записей;
- 6) *Доставка во внешний контур.* Тот же сервис принимает со-

общения из очереди `integration.webhook.v1`, выполняет HTTP-доставку внешнему получателю и фиксирует попытки в `delivery_attempts`;

- 7) *Контроль и наблюдаемость.* Все сервисы экспортируют метрики Prometheus и журналы, на основе которых рассчитываются эксплуатационные показатели платформы;
- 8) *Операторские сценарии.* Сервис пользовательского интерфейса предоставляет роль-зависимый интерфейс для мониторинга камер, журнала событий и аналитических отчетов.

Описанная логика непосредственно соответствует формальному определению конвейера Φ из раздела 2.1 и связывает обоснованный технологический стек с математической моделью обработки.

2.3 Модель детекции объектов и фильтрации классов

2.3.1 Выход детектора и базовые обозначения

Для кадра $\hat{x}_t$ детектор возвращает множество боксов

$$\mathcal{D}_t = \{d_k\}_{k=1}^{K_t}, \quad d_k = (b_k, c_k, p_k), \quad (8)$$

где $b_k = (x_k, y_k, w_k, h_k)$ — прямоугольник локализации объекта;

c_k — класс обнаруженного объекта;

$p_k \in [0, 1]$ — оценка достоверности детекции.

Для подавления шумовых кандидатов используется порог

$$\mathcal{D}_t^{(\tau)} = \{d_k \in \mathcal{D}_t \mid p_k \geq \tau_{\text{conf}}\}. \quad (9)$$

После пороговой фильтрации применяется NMS по IoU:

$$\text{IoU}(b_a, b_b) = \frac{|b_a \cap b_b|}{|b_a \cup b_b|}, \quad (10)$$

и кандидаты с $\text{IoU} > \tau_{\text{iou}}$ подавляются в пользу бокса с большим p_k [27; 30].

2.3.2 Фильтрация целевых классов

Множество целевых классов не является жестко зашитой константой алгоритма и задается профилем применения платформы:

$$\mathcal{C}_{\text{target}}^{(p)} \subseteq \mathcal{C}_{\text{model}}, \quad (11)$$

где p — профиль применения платформы;

$\mathcal{C}_{\text{model}}$ — множество классов, поддерживаемых подключенной моделью детекции.

Для базового охранного профиля используется множество

$$\mathcal{C}_{\text{target}}^{(\text{security})} = \{\text{person}, \text{vehicle}\}, \quad (12)$$

где `vehicle` — агрегированный класс транспорта (`car`, `bus`, `truck`, `motorcycle`) на уровне API-конфигурации.

Для демонстрационного профиля обнаружения БПЛА используется другое множество целевых классов:

$$\mathcal{C}_{\text{target}}^{(\text{uav})} = \{\text{bird}, \text{drone}\}. \quad (13)$$

Итоговое множество детекций, передаваемых в модуль сопровождения для выбранного профиля p :

$$\mathcal{D}_t^*(p) = \{d_k \in \mathcal{D}_t^{(\tau)} \mid c_k \in \mathcal{C}_{\text{target}}^{(p)}\}. \quad (14)$$

Выбор CPU-first профиля для разработки означает, что τ_{conf} , входное разрешение и частота инференса подбираются так, чтобы сохранять ограничение по задержке в целевых сценариях [28].

2.3.3 Оценка качества детекции

Для оффлайн-оценки параметров детектора вводится стандартная метрика среднего качества локализации:

$$\text{mAP} = \frac{1}{|\mathcal{C}_{\text{target}}|} \sum_{c \in \mathcal{C}_{\text{target}}} \text{AP}_c, \quad (15)$$

где AP_c — площадь под кривой precision-recall для класса c при заданных порогах IoU.

Однако в рамках платформы реального времени метрики детекции анализируются совместно с показателями последующих этапов: устойчивостью траекторий и точностью генерации событий. Это исключает ситуацию, когда рост mAP не приводит к улучшению эксплуатационного качества контура.

2.4 Модель сопровождения объектов

2.4.1 Состояние трека и прогноз

Состояние трека j в модели DeepSORT задается вектором

$$\mathbf{s}_{j,t} = [u, v, \gamma, h, \dot{u}, \dot{v}, \dot{\gamma}, \dot{h}]^\top, \quad (16)$$

где (u, v) — центр бокса;

γ — отношение сторон бокса;

h — высота бокса;

$\dot{u}, \dot{v}, \dot{\gamma}, \dot{h}$ — скорости изменения соответствующих компонент.

[2]

Прогноз состояния выполняется линейной динамической моделью (фильтр Калмана):

$$\mathbf{s}_{j,t|t-1} = \mathbf{F}\mathbf{s}_{j,t-1|t-1} + \mathbf{w}_t, \quad (17)$$

где $\mathbf{w}_t$ — шум процесса.

2.4.2 Ассоциация детекций и траекторий

Для пары «траектория--детекция» рассчитывается композиционная стоимость:

$$\mathcal{L}(j, k) = \lambda \cdot d_{\text{motion}}(j, k) + (1 - \lambda) \cdot d_{\text{app}}(j, k), \quad (18)$$

где d_{motion} — расстояние Махаланобиса относительно прогноза;

d_{app} — косинусная дистанция признаков внешнего вида.

Оптимальное сопоставление выполняется венгерским алгоритмом [29]. Пары с превышением порогов по d_{motion} или d_{app} отбрасываются.

2.4.3 Устойчивость идентификаторов и fallback-логика

Практическая устойчивость модуля сопровождения оценивается метриками *ID-switches*, *MOTA*, *MOTP* и долей прерванных траекторий [11; 12].

Для повышения устойчивости в контуре исполнения применяется fallback-логика:

- 1) при кратковременной потере детекции траектория удерживается в состоянии *tentative/lost* до таймаута T_{lost} ;
- 2) при превышении T_{lost} траектория закрывается, но ее финальные атрибуты сохраняются для правил *dwel_time*;
- 3) при деградации *aprearance*-эмбедингов вес λ динамически смещается в сторону кинематической составляющей.

Изоляция состояния по камерам формально задается как

$$\mathcal{T}_t = \bigsqcup_{i=1}^N \mathcal{T}_t^i, \quad (19)$$

то есть объединение траекторий без пересечения пространств идентификаторов между различными камерами.

2.5 Правиловая модель событий и дедупликация

2.5.1 Событие как предикат над траекторией

Каждое правило $r \in \mathcal{R}_i$ задает предикат

$$P_r(\tau_j, \mathcal{Z}_i, t) \in \{0, 1\}. \quad (20)$$

Если $P_r = 1$, создается событие

$$e = (\text{event_id}, \text{event_type}, \text{camera_id}, \text{track_id}, \text{occurred_at}, \text{severity}, \text{dedup_key}, \dots). \quad (21)$$

2.5.2 Правило `zone_enter`

Для полигона зоны Z и траектории τ_j используется предикат входа:

$$P_{\text{zone_enter}} = \mathbb{K}\{p_{t-1} \notin Z \wedge p_t \in Z\}, \quad (22)$$

где p_t — реперная точка объекта (центр нижней грани бокса).

2.5.3 Правило `line_cross`

Для ориентированной линии $L = (a, b)$ вычисляется знак функции ориентации:

$$\sigma_t = \text{sign}((b - a) \times (p_t - a)). \quad (23)$$

Факт пересечения фиксируется при смене знака $\sigma_{t-1}\sigma_t < 0$ с учетом минимального смещения по времени, чтобы подавить дрожание около линии.

2.5.4 Правило `dwell_time`

Для зоны Z и порога $\Delta T_{\min}$ правило задается:

$$P_{\text{dwell}} = \mathbb{K}\{p_t \in Z \wedge (t - t_{\text{enter}}) \geq \Delta T_{\min}\}. \quad (24)$$

Событие формируется однократно при первом выполнении условия.

2.5.5 Модель `dedup_key`

Для подавления повторной генерации вводится функция

$$\kappa(e) = \text{camera_id} \parallel \text{rule_id} \parallel \text{track_id} \parallel \text{bucket}(t), \quad (25)$$

где $\text{bucket}(t)$ — опциональный временной бакет для правил длительности.

Если ключ $\kappa(e)$ уже присутствует в скользящем окне дедупликации, событие в таблицу исходящих сообщений не добавляется. Формально:

$$\text{emit}(e) = \mathbb{K}\{\kappa(e) \notin \mathcal{K}_{\text{window}}\}. \quad (26)$$

2.6 Модель надежной доставки: исходящие сообщения, повторы и очередь недоставленных сообщений

2.6.1 Транзакционная фиксация события

Пусть транзакция T вставляет записи в таблицы `events` и `event_outbox`. Гарантия атомарности:

$$\text{commit}(T) \Rightarrow (\exists e \in \text{events}) \wedge (\exists o \in \text{event_outbox} : o.\text{event_id} = e.\text{event_id}). \quad (27)$$

При откате транзакции (`rollback`) обе записи отсутствуют [3; 13].

2.6.2 Повторные попытки публикации

Для записи таблицы исходящих сообщений со счетчиком попыток n время следующей отправки:

$$t_{\text{next}} = t_0 + \min(t_{\text{max}}, t_{\text{base}} \cdot 2^n). \quad (28)$$

Политика ограничивается $n \leq 10$, после чего запись переводится в конечное состояние ошибки.

2.6.3 Модель потребителя и очереди недоставленных сообщений

Интеграционный потребитель очереди HTTP-доставки обрабатывает сообщение и отправляет брокеру подтверждение обработки (АСК) только после успешной доставки HTTP-получателю. При исчерпании лимита попыток сообщение маршрутизируется в очередь недоставленных сообщений [14; 31].

Для прикладной надежности используется приближенная оценка вероятности успешной доставки:

$$P_{\text{succ}} = 1 - \prod_{k=1}^n (1 - p_k), \quad (29)$$

где p_k — вероятность успешной доставки на k -й попытке.

Рост n увеличивает P_{succ} , но повышает задержку и нагрузку на внешний контур.

2.7 Оценка вычислительной сложности и ресурсной нагрузки

2.7.1 Сложность стадий конвейера

Для практического применения важно оценить не только формальную корректность модели, но и ее вычислительную стоимость. Введем обозначения:

- F_i - входной FPS камеры i ;
- K_t - число детекций в кадре t ;
- M_t - число активных траекторий;
- R - число правил для камеры.

Стадия захвата и декодирования имеет стоимость, пропорциональную числу декодируемых кадров:

$$\mathcal{O}_{\text{capture}} \sim \mathcal{O}(F_i). \quad (30)$$

При политике замещения последним актуальным кадром часть кадров отбрасывается до инференса, что снижает нагрузку на последующие этапы конвейера.

Стадия детекции имеет наибольшую вычислительную стоимость и определяется архитектурой модели и входным разрешением. В инженерной оценке ее удобно учитывать через измеряемую задержку Δt_{infer} , а не через аналитическую асимптотику.

Для сопровождения ключевым является шаг ассоциации «детекции--треки». При использовании венгерского алгоритма верхняя оценка:

$$\mathcal{O}_{\text{assoc}} \sim \mathcal{O}(\max(K_t, M_t)^3). \quad (31)$$

На практике после гейтинга и фильтрации матрица стоимости разрежена, поэтому фактическая стоимость ниже.

Стоимость правилowego движка в базовой реализации:

$$\mathcal{O}_{\text{rules}} \sim \mathcal{O}(M_t \cdot R), \quad (32)$$

поскольку каждая активная траектория проверяется по набору правил камеры.

2.7.2 Задержка сквозного контура

Суммарная задержка от объекта к событию представляется как

$$\Delta t_{\text{obj} \rightarrow \text{event}} = \Delta t_{\text{capture}} + \Delta t_{\text{queue}} + \Delta t_{\text{infer}} + \Delta t_{\text{track}} + \Delta t_{\text{rules}} + \Delta t_{\text{store}}. \quad (33)$$

Если очередь между захватом и инференсом не ограничена, Δt_{queue} может расти неограниченно. В предлагаемой модели размер очереди фиксирован единицей, поэтому:

$$\Delta t_{\text{queue}} \leq \frac{1}{F_i}, \quad (34)$$

что гарантирует отсутствие лавинообразного накопления задержки.

2.7.3 Ресурсные ограничения на одном узле

Односерверная модель требует явного баланса между числом камер и доступными ресурсами:

- центральный процессор (CPU): декодирование, сопровождение объектов, правила, сериализация событий;
- аппаратный ускоритель: инференс детектора при соответствующем стенде;
- оперативная память (RAM): буферы кадров, состояния траекторий, кэш конфигурации;
- дисковый ввод-вывод: запись событий, таблицы исходящих сообщений и артефактов.

Для камеры i можно использовать приближенную модель использования памяти:

$$Mem_i \approx Mem_{\text{frame}} + Mem_{\text{tracks}} + Mem_{\text{exec}}, \quad (35)$$

где Mem_{frame} — память, занимаемая кадровым буфером;

Mem_{tracks} — память состояния активных траекторий;

Mem_{exec} — накладные расходы модели и библиотек в контуре исполнения.

Эта оценка используется для первичной оценки емкости стенда перед приемочными прогонами.

2.8 Псевдокод целевого алгоритма

2.8.1 Алгоритм обработки одной камеры

Ниже приведен укрупненный алгоритм контура исполнения для камеры i :

- 1) Инициализировать источник потока, модуль сопровождения и локальный кэш правил.
- 2) Пока камера активна:
 - a) Получить очередной кадр из RTSP/MP4.
 - b) Нормализовать временную метку и записать кадр в буфер последнего актуального кадра (замещая предыдущий).
 - c) Если есть актуальный кадр для инференса, запустить детектор.
 - d) Отфильтровать детекции по порогам достоверности и IoU, а также по целевым классам.
 - e) Обновить модуль сопровождения и получить список активных траекторий.
 - f) Проверить правила `zone_enter`, `line_cross`, `dwel_time`.
 - g) Для каждого нового факта сформировать `dedup_key`.
 - h) Если ключ не встречался в окне дедупликации, в одной транзакции записать:
 - i) запись в `events`;
 - ii) запись в `event_outbox`.

Такое представление соответствует архитектурной декомпозиции из главы 3 и позволяет связать математическую модель с кодовой реализацией модулей.

2.8.2 Алгоритм работы публикатора исходящих сообщений

- 1) Выбрать ограниченную партию записей таблицы исходящих сообщений со статусом `pending/retry`.
- 2) Для каждой записи:
 - a) Сформировать ключ маршрутизации вида `event.<type>.v1`.
 - b) Опубликовать сообщение в обменник `events.topic.v1`.
 - c) При подтверждении публикации отметить запись как

published.

- d) При ошибке увеличить `retry_count` и вычислить `next_retry_at` по политике задержки между повторами.

3) Повторять цикл с малым интервалом опроса.

2.8.3 Алгоритм доставки HTTP-получателю

- 1) Получить сообщение из очереди `integration.webhook.v1`.
- 2) Выполнить HTTP POST во внешний конечный адрес.
- 3) Зафиксировать попытку в `delivery_attempts`.
- 4) При успехе отправить брокеру подтверждение обработки (ACK).
- 5) При ошибке:
 - a) если лимит попыток не исчерпан, инициировать повтор;
 - b) иначе отправить сообщение в очередь недоставленных сообщений и пометить конечную ошибку доставки.

2.9 Параметризация и калибровка модели

2.9.1 Набор настраиваемых параметров

Для воспроизводимых экспериментов фиксируется конфигурационный вектор:

$$\Theta = \{\tau_{\text{conf}}, \tau_{\text{iou}}, \lambda, T_{\text{lost}}, \Delta T_{\text{min}}, n_{\text{max}}, t_{\text{base}}, t_{\text{max}}\}. \quad (36)$$

где τ_{conf} — порог достоверности детекции;

τ_{iou} — порог IoU для NMS;

λ — баланс признаков движения и внешнего вида в ассоциации;

T_{lost} — таймаут удержания потерянного трека;

ΔT_{min} — порог для `dwelling_time`;

n_{max} — максимальное число повторных попыток;

t_{base} — базовый интервал между повторами;

t_{max} — максимальный интервал между повторами.

2.9.2 Принцип калибровки

Калибровка выполняется поэтапно:

- 1) подобрать $\tau_{\text{conf}}, \tau_{\text{iou}}$ на валидационном подмножестве кадров;

- 2) зафиксировать параметры модуля сопровождения (λ, T_{lost}) по критерию минимизации $ID\text{-switches}$;
- 3) настроить пороги правил так, чтобы снизить ложные события без роста пропусков;
- 4) выбрать параметры повторной доставки и задержки между повторами по компромиссу «надежность---задержка».

Важный практический момент: оптимум по одной метрике (например, mAP детектора) не гарантирует оптимум по системной метрике $p95_{\text{obj} \rightarrow \text{event}}$. Поэтому окончательная калибровка должна проводиться в целевом видео-контуре, а текущая работа фиксирует модель такой калибровки и границы стендовой проверки.

2.9.3 Ограничения валидности модели

Текущая формализация имеет осознанные ограничения:

- нет межкамерной реидентификации;
- правилый движок ориентирован на атомарные события и не покрывает сложные составные сценарии;
- оценки производительности чувствительны к характеристикам видео-потока и стенда.

Несмотря на это, модель достаточна для целей текущего исследования и для проведения корректной экспериментальной оценки, предусмотренной главой 4.

2.10 Метрики качества подсистем и их агрегация

2.10.1 Метрики детекции

Для детектора в offline-контуре фиксируются:

- AP_{50}, AP_{75} ;
- $mAP_{50:95}$;
- precision/recall по целевым классам.

При переносе в online-контур детектор оценивается не только по точности, но и по Δt_{infer} , поскольку именно эта величина напрямую влияет на $p95_{\text{obj} \rightarrow \text{event}}$.

2.10.2 Метрики сопровождения

Для модуля сопровождения используются метрики:

- *MOTA* - агрегированная точность сопровождения;
- *MOTP* - точность локализации в сопровождении;
- *IDF1* - качество сохранения идентичности;
- *ID-switches* - число переключений ID.

Для задач охрannого мониторинга ключевыми являются *IDF1* и *ID-switches*, поскольку они определяют корректность событий, зависящих от длительности и направления движения объекта.

2.10.3 Метрики событийного уровня

Событийный модуль оценивается как задача бинарной классификации фактов:

$$\text{precision}_e = \frac{TP_e}{TP_e + FP_e}, \quad \text{recall}_e = \frac{TP_e}{TP_e + FN_e}. \quad (37)$$

Дополнительно фиксируется доля подавленных дубликатов:

$$R_{\text{dedup}} = \frac{N_{\text{suppressed}}}{N_{\text{raw_triggers}}}. \quad (38)$$

Слишком низкое R_{dedup} может означать недоиспользование механизма дедупликации, слишком высокое - риск агрессивного подавления валидных событий.

2.10.4 Метрики надежности интеграции

Для контура доставки:

$$R_{\text{publish}} = \frac{N_{\text{published}}}{N_{\text{outbox}}}, \quad R_{\text{webhook}} = \frac{N_{\text{webhook_ok}}}{N_{\text{published}}}. \quad (39)$$

Также учитываются:

- среднее число повторных попыток на событие;
- доля сообщений, переведенных в очередь недоставленных сообщений;
- время обработки накопленной очереди исходящих сообщений после восстановления брокера.

2.10.5 Сквозная агрегированная оценка

Для интегральной оценки качества платформы вводится вектор

$$\mathbf{Q} = [\text{p95}_{\text{obj} \rightarrow \text{event}}, IDF1, \text{precision}_e, R_{\text{webhook}}, R_{\text{drop}}]. \quad (40)$$

Решение считается приемлемым, если каждая компонента $\mathbf{Q}$ удовлетворяет целевому порогу. Такой подход предпочтительнее единой скалярной функции, поскольку позволяет локализовать конкретный деградирующий слой конвейера.

2.11 Формализация ограничений и критериев приемки

2.11.1 Ограничения по времени

Пороговые значения в работе задаются как инженерные критерии стендовой приемки: они не подменяют сертификационные процедуры, но согласуются с логикой измерения производительности интеллектуальной видеоаналитики в системах видеонаблюдения [5]. Для CPU-профиля:

$$\text{p95}_{\text{obj} \rightarrow \text{event}} \leq 2.0 \text{ с.} \quad (41)$$

Для расширенных многокамерных профилей пороги должны устанавливаться только после отдельной серии измерений на целевом оборудовании. Нарушение выбранных ограничений требует пересмотра параметров Θ или масштабирования вычислительных ресурсов.

2.11.2 Ограничения по надежности

Требование надежности формализуется как:

$$N_{\text{lost}} = 0 \quad \text{при кратковременной недоступности RabbitMQ,} \quad (42)$$

а также

$$\frac{N_{\text{dlq}}}{N_{\text{generated}}} \leq \epsilon_{\text{dlq}}, \quad (43)$$

где ϵ_{dlq} — допустимый порог доли сообщений, переведенных в очередь недоставленных сообщений.

2.11.3 Ограничения по безопасности

Для защищенных операций API должна выполняться аксиома разграничения доступа, согласованная с принципами минимально необходимых привилегий, управления доступом и подотчетности действий [16]:

$$\forall u \in \mathcal{U}, \forall a \in \mathcal{A}_{\text{protected}} : \text{permit}(u, a) \Leftrightarrow \text{role}(u) \in \mathcal{R}(a). \quad (44)$$

Нарушение данной аксиомы трактуется как критическая дефектность контура безопасности и блокирует приемку релиза.

2.11.4 Критерий архитектурной приемки

Финальный критерий приемки задается множеством ограничений:

$$\mathcal{C} = \{C_{\text{time}}, C_{\text{reliability}}, C_{\text{security}}, C_{\text{observability}}\}. \quad (45)$$

Платформа считается прошедшей архитектурную приемку, если:

$$\bigwedge_{c \in \mathcal{C}} c = 1. \quad (46)$$

где C_{time} — соблюдены ограничения по задержке;

$C_{\text{reliability}}$ — подтверждена корректность таблицы исходящих сообщений, повторов и очереди недоставленных сообщений;

C_{security} — RBAC-ограничения подтверждены тестами;

$C_{\text{observability}}$ — все обязательные метрики доступны.

Данный критерий используется как формальная основа интерпретации результатов в главе 4.

Выводы по главе 2

Во второй главе сформированы теоретические предпосылки, модели и методология построения платформы видеоаналитики реального времени. Основные результаты главы могут быть резюмированы тремя положениями.

Во-первых, обоснован выбор архитектурного шаблона и технологического стека. Микросервисная архитектура с разделением на контур управления и контур обработки данных согласована с архитектурными факторами,

сформулированными в главе 1 (изоляция отказов, локальность изменений, наблюдаемость). Для каждого слоя стека (детектор, модуль сопровождения, веб-фреймворк, СУБД, брокер сообщений, видеозахват, наблюдаемость, оркестрация) выполнено сравнение альтернатив и зафиксирован обоснованный выбор: YOLO [1; 27; 28] и DeepSORT [2; 8; 29] для алгоритмического ядра, FastAPI [22] и SQLAlchemy [23] для управляющего сервиса, PostgreSQL [32] и RabbitMQ [14; 24; 31] для надежной фиксации и асинхронной доставки событий, Prometheus и Grafana [7; 15] для наблюдаемости, Docker Compose [25] для воспроизводимого стенда. Сводный технологический стек отражен в таблице 2.6, его архитектурное представление - на рисунках 2.1 и 2.2.

Во-вторых, сформирована математическая модель сквозного конвейера платформы: от захвата видеопотока и детекции до сопровождения, правильной генерации событий и надежной доставки через транзакционную таблицу исходящих сообщений [3; 13]. Зафиксированы формальные определения для трех базовых правил (`zone_enter`, `line_cross`, `dwell_time`), введена модель дедупликации на основе `dedup_key`, описаны политика повторной доставки с экспоненциальной задержкой между попытками и маршрутизация в очередь недоставленных сообщений. Диаграмма последовательности доставки события (рисунок 2.3) связывает этапы транзакционной фиксации и асинхронной публикации с конкретными участниками контура.

В-третьих, введены оценки вычислительной сложности этапов конвейера и формализован критерий архитектурной приемки $\bigwedge_{c \in \mathcal{C}} c = 1$, объединяющий ограничения по времени, надежности, безопасности и наблюдаемости. Зафиксированы наборы метрик подсистем (детекции, сопровождения, событийного уровня, надежности интеграции) и сводный вектор интегральной оценки **Q**. Полученные формализации используются как методическая основа инженерной реализации платформы (глава 3) и как набор критериев для интерпретации результатов; в главе 4 фактические измерения отделяются от целевых критериев расширенной приемки.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Архитектура сервисов и инфраструктурный контур

Реализация выполнена как набор изолированных микросервисов в одном репозитории:

- `api` - управление конфигурацией, аутентификация/авторизация, просмотр событий, подтверждение инцидентов и экспорт подтверждающих артефактов;
- `analytics-worker` - обработка видеопотоков и формирование событий;
- `integration-worker` - публикация записей `event_outbox` в RabbitMQ и доставка HTTP-получателям;
- `ui` - операторский интерфейс;
- инфраструктурные компоненты: `postgres`, `rabbitmq`, `prometheus`, `grafana`, `webhook-mock`.

Такое разбиение обеспечивает независимые жизненные циклы сервисов при сохранении простоты развёртывания на одном сервере через Docker Compose [24; 25; 32].

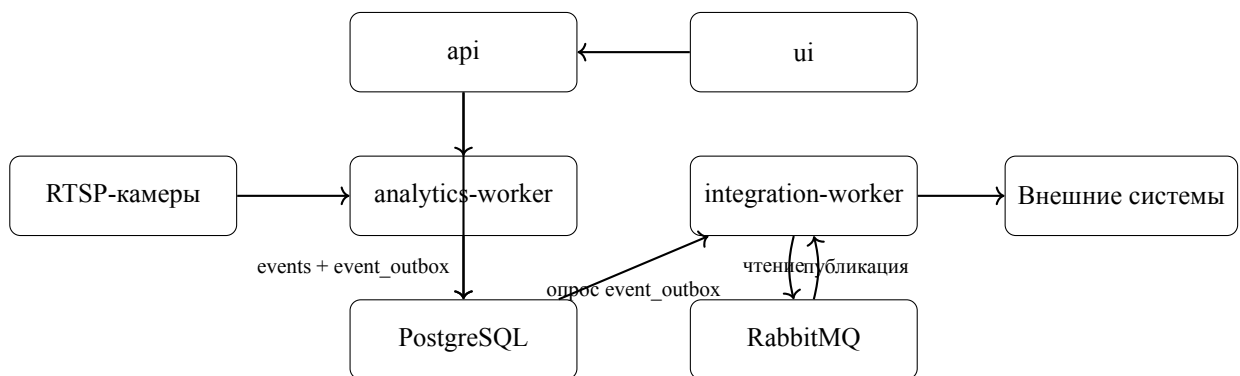


Рисунок 3.1 - Схема взаимодействия сервисов платформы

Таблица 3.1 - Сервисы и зоны ответственности

Сервис	Основная ответственность
api	Операции CRUD для камер, зон и правил, журнал событий, маршруты подтверждения и артефактов, JWT/RBAC, маршруты состояния и метрик
analytics-worker	Захват RTSP/MP4, детекция, сопровождение объектов, правилowej движок, запись events + event_outbox
integration-worker	Публикация в обменник events.topic.v1, доставка HTTP-получателям, повторы, очередь недо-ставленных сообщений, журнал попыток доставки
ui	Вход пользователя, управление конфигурацией, просмотр событий, статусы системы
prometheus + grafana	Сбор и визуализация эксплуатационных метрик

3.2 Реализация программного интерфейса (API), JWT/RBAC и контрактов

3.2.1 Технологический стек программного интерфейса

Сервис программного интерфейса (API) реализован на FastAPI с асинхронной моделью обработки запросов, ORM-слоем SQLAlchemy и миграциями Alembic [22; 23; 39]. Для валидации входных данных используются Pydantic-схемы; для интеграционных контрактов зафиксирована OpenAPI-спецификация.

3.2.2 Модель пользователей и ролей

В БД реализованы сущности:

- users;
- roles;
- user_roles;
- refresh_tokens.

Поддерживаются роли `admin`, `operator`, `viewer`. Ограничения на маршруты реализованы через слой зависимостей проверки прав; такая декомпозиция согласуется с практикой явного разграничения доступа и последующего аудита действий пользователей [16].

Таблица 3.2 - Ролевая матрица доступа

Операция	admin	operator	viewer
CRUD пользователей	+	-	-
CRUD камер/зон/правил	+	+	-
Просмотр событий и статусов	+	+	+
Метрики и маршруты состояния	+	+	+

3.2.3 Сценарий JWT-аутентификации

Контур безопасности использует пару токенов:

- маркер доступа `access token` (TTL 15 минут);
- маркер обновления `refresh token` (TTL 7 дней).

Маркеры обновления хранятся в БД в хэшированном виде и отзываются при завершении сеанса `logout`. Это позволяет централизованно управлять активными сессиями и реализовать принудительный отзыв.

3.2.4 Контракты REST API

Базовые маршруты:

- авторизация: `/api/v1/auth/login`, `/api/v1/auth/refresh`;
- сессии: `/api/v1/auth/logout`, `/api/v1/auth/me`;
- конфигурация: `/api/v1/cameras`, `/api/v1/zones`;
- правила: `/api/v1/rules`;
- журнал событий: `/api/v1/events`;
- действия оператора по обработке событий: `PATCH/api/v1/events/{event_id}/ack` и `GET/api/v1/events/{event_id}/evidence`;
- эксплуатационные маршруты: `/health/live`, `/health/ready`,

`/metrics.`

Для геометрии зон и линий используется JSONB-представление в формате, близком к GeoJSON, с серверной валидацией структуры и допустимости координат.

3.3 Реализация `analytics-worker`

3.3.1 Захват и декодирование видеопотока

Контур захвата построен на PyAV/FFmpeg, что позволяет управлять буферизацией, переключением и обработкой нестабильных RTSP-источников [9; 10]. Для воспроизводимых тестов поддержан резервный режим работы с MP4-файлами.

На каждую камеру выделяется независимый контекст обработки:

- источник потока;
- очередь последнего кадра размером 1;
- состояние сопровождения;
- локальные счетчики метрик.

3.3.2 Детекция и сопровождение объектов

Детекция выполняется моделью семейства YOLO, имя весов задается параметром `DETECTOR_MODEL`. В коде аналитического сервиса штатное значение параметра оставлено как легкий общий профиль `yolov8n.pt`; в экспериментальном БПЛА-профиле этот параметр переопределяется на дообученные веса `best.pt`, полученные из начального варианта `yolo26n.pt`. Сопровождение объектов реализовано через адаптер к `deep-sort-realtime` с камерной изоляцией состояния [2; 40].

Горячий путь:

$$\begin{aligned} \text{захват} &\rightarrow \text{буфер кадра} \rightarrow \text{инференс} \\ &\rightarrow \text{сопровождение} \rightarrow \text{правила.} \end{aligned} \tag{1}$$

Это исключает неконтролируемый рост задержки даже при временных всплесках входного FPS.

3.3.3 Правилowej движок и генерация событий

Реализованы правила:

- zone_enter;
- line_cross;
- dwell_time.

Событие формируется только при выполнении предиката и прохождении дедупликации через `dedup_key`. Далее в рамках одной транзакции записываются:

- 1) запись в `events`;
- 2) запись в `event_outbox`.

Таким образом устраняется гонка между «событие зафиксировано» и «событие опубликовано» [3; 13].

Фрагмент реализации правила входа в зону приведен в листинге 3.1. В коде явно сравниваются предыдущее и текущее положения центра bbox: событие создается только при переходе из состояния «вне зоны» в состояние «внутри зоны», а `dedup_key` связывает камеру, правило и траекторию.

Листинг 3.1: Фрагмент реализации правила `zone_enter`

```
1 if rule_type == "zone_enter" and zone:
2     polygon = (zone.get("geometry") or {}).get("points", [])
3     inside_now = point_in_polygon(current_point, polygon)
4     inside_prev = (
5         point_in_polygon(st.last_point, polygon)
6         if st.last_point
7         else False
8     )
9     if inside_now and not inside_prev:
10         dedup = f"{self.camera_id}:{rule['rule_id']}:{track_id}"
11         event = self._build_event(
12             track=track,
13             rule=rule,
14             event_type="zone_enter",
15             dedup_key=dedup,
16             zone_id=zone_id,
17         )
18         generated.append(event)
```

3.4 Реализация `integration-worker`: RabbitMQ, повторы и очередь недоставленных сообщений

3.4.1 Топология RabbitMQ

Интеграционный контур использует:

- устойчивый обменник `events.topic.v1` (тип `topic`);
- устойчивую очередь `integration.webhook.v1`;
- ключ маршрутизации формата `event.<event_type>.v1`;
- `integration.webhook.v1.dlq` - очередь недоставленных сообщений.

Публикация из таблицы исходящих сообщений в брокер выполняется с подтверждениями публикации RabbitMQ; потребитель подтверждает сообщение (`ack`) только после успешного вызова HTTP-получателя [14; 31].

3.4.2 Публикатор исходящих сообщений

Публикатор выбирает записи таблицы исходящих сообщений в статусах `pending/retry`, сериализует полезную нагрузку и публикует ее в обменник. После получения подтверждения статус переводится в `published`; при ошибке увеличивается `retry_count` и пересчитывается `next_retry_at`.

Листинг 3.2: Цикл публикации исходящих сообщений с контролируемыми повторами

```
1 for row in candidates:
2     outbox_id = row["outbox_id"]
3     retry_count = int(row.get("retry_count", 0))
4     payload = row["payload"]
5     event_type = payload.get("event_type", "unknown")
6     try:
7         broker.publish_event(payload, event_type)
8         mark_published(outbox_id)
9         PUBLISH_SUCCESS.inc()
10    except Exception:
11        retry_count += 1
12        terminal = retry_count >= settings.max_retry
13        mark_retry(outbox_id, retry_count, terminal=terminal)
14        PUBLISH_ERROR.inc()
15        if not terminal:
16            OUTBOX_RETRIES.inc()
17            reconnect_required = True
18        break
```

3.4.3 Доставка HTTP-получателю и журнал попыток

Потребитель выполняет POST-запрос целевому HTTP-получателю (в тестовом контуре - webhook-mock) и записывает результат в `delivery_attempts`. Стратегия повторов использует экспоненциальное увеличение задержки до 10 попыток, затем фиксируется конечная ошибка доставки и сообщение маршрутизируется в очередь недоставленных сообщений.

Для внешних систем поддержана идемпотентность на уровне `dedup_key`: повторная доставка одного факта не должна приводить к дублирующим побочным эффектам.

Фрагмент доставки HTTP-получателю приведен в листинге 3.3. Потребитель подтверждает сообщение брокеру только после успешной доставки либо после контролируемого перевода в очередь недоставленных сообщений, что исключает бесконечный цикл повторов без фиксации причины отказа.

Листинг 3.3: Доставка HTTP-получателю с повтором и переходом в очередь недоставленных сообщений

```
1 success, error_text = _deliver_webhook(payload)
2 if success:
3     WEBHOOK_SUCCESS.inc()
4     log_delivery_attempt(event_id, consumer_name, "success", None)
5     channel.basic_ack(delivery_tag=method.delivery_tag)
6     return
7
8 WEBHOOK_FAILURE.inc()
9 log_delivery_attempt(event_id, consumer_name, "failed", error_text)
10 next_attempt = attempt + 1
11 if next_attempt >= settings.max_retry:
12     broker.publish_dlq(payload, reason=error_text or "max_retries")
13     DLQ_PUBLISH.inc()
14     channel.basic_ack(delivery_tag=method.delivery_tag)
15     return
16
17 time.sleep(min(30, 2**next_attempt))
18 broker.publish_event(
19     payload,
20     payload.get("event_type", "unknown"),
21     headers={"delivery_attempt": next_attempt},
22 )
23 OUTBOX_RETRIES.inc()
24 channel.basic_ack(delivery_tag=method.delivery_tag)
```


3.5 Мониторинг, метрики и эксплуатационная наблюдаемость

Наблюдаемость реализована как обязательная часть решения через Prometheus и Grafana [7; 15].

Сервисы публикуют:

- /health/live;
- /health/ready;
- /metrics.

Ключевые метрики:

- входной и обработанный FPS по камерам;
- количество отброшенных кадров;
- задержка инференса;
- задержка `object->event`;
- число reconnect к RTSP;
- ошибки и повторы публикации исходящих сообщений;
- неуспешные доставки и счетчик сообщений в очереди недоставленных сообщений.

Фиксация этих показателей позволяет в главе 4 перейти от декларативной оценки к измеримой.

3.6 Развертывание и воспроизводимость

3.6.1 Docker Compose профиль

Для единого стенда используется `docker-compose.yml`, включающий полный набор сервисов текущего контура. Развертывание воспроизводимо как локально, так и на выделенном Linux-сервере.

3.6.2 Makefile и автоматизация

На уровне репозитория предусмотрены команды:

- управление стеком (`up`, `down`, `logs`);
- тестовые цели (`test`, `ui-test`, `smoke-e2e`);
- цели для отчета (`report-pdf`, `report-clean`, `report-check`).

Для LaTeX-сборки по умолчанию используется контейнерный конвейер-

ер TeX Live `xelatex -> biber -> xelatex -> xelatex`. Это устраняет зависимость от локальной установки TeX на рабочей машине.

3.7 Реализация операторского интерфейса (UI)

3.7.1 Структура клиентского приложения

Пользовательский интерфейс реализован как одностраничное приложение на React + Vite + TypeScript. Приложение разделено на следующие функциональные модули:

- модуль аутентификации и управления сессией;
- модуль конфигурации камер;
- модуль управления зонами и правилами;
- журнал событий с фильтрацией, подтверждением и экспортом подтверждающих артефактов;
- страница статуса системы и метрик.

Маршруты интерфейса соответствуют принятым контрактам:

```
/login, /dashboard, /cameras, /zones,  
/rules, /events, /demo, /status. (2)
```

3.7.2 Сценарий JWT-аутентификации на стороне интерфейса

При успешном входе пользовательский интерфейс получает маркеры доступа и обновления и выполняет:

- 1) хранение маркера доступа в памяти приложения;
- 2) хранение маркера обновления в защищенном хранилище сессии (в соответствии с политикой проекта);
- 3) автоматическую ротацию маркера доступа при получении ответа 401.

Завершение сеанса инициирует отзыв маркера обновления на стороне API и локальную очистку сессионных данных.

3.7.3 Ролевая видимость действий

В интерфейсе применяется не только серверное ограничение, но и визуальная фильтрация действий:

- viewer: доступ к чтению событий, статусов и метрик;
- operator: доступ к изменению камер/зон/правил;
- admin: доступ к управлению пользователями и ролями.

Это уменьшает вероятность операторских ошибок и делает интерфейс согласованным с RBAC-политикой API.

Контрольная панель оператора агрегирует состояние камер, событий и сервисов, что позволяет использовать UI как единое рабочее место дежурной смены (рисунок 3.2).

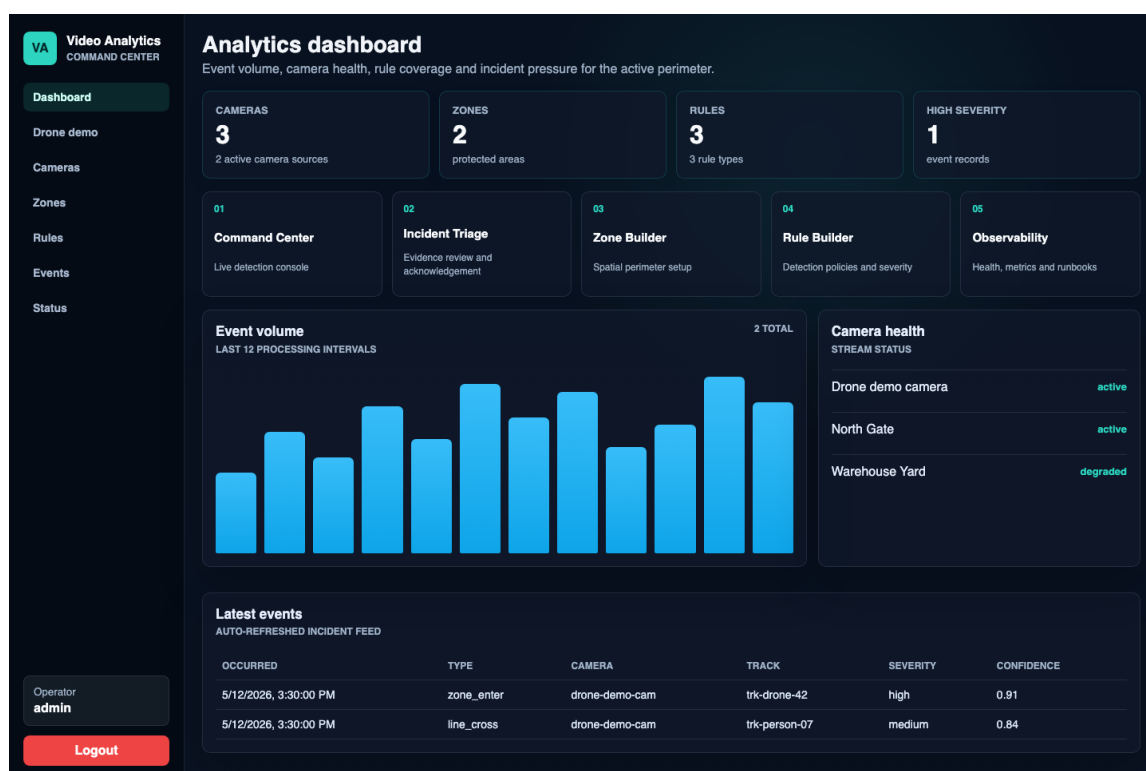


Рисунок 3.2 - Экран сводной панели оператора

Экран демонстрации БПЛА объединяет видеопоток, активные траектории, события и состояние правила, что позволяет показать прохождение результата детекции в прикладной контур платформы (рисунок 3.3).

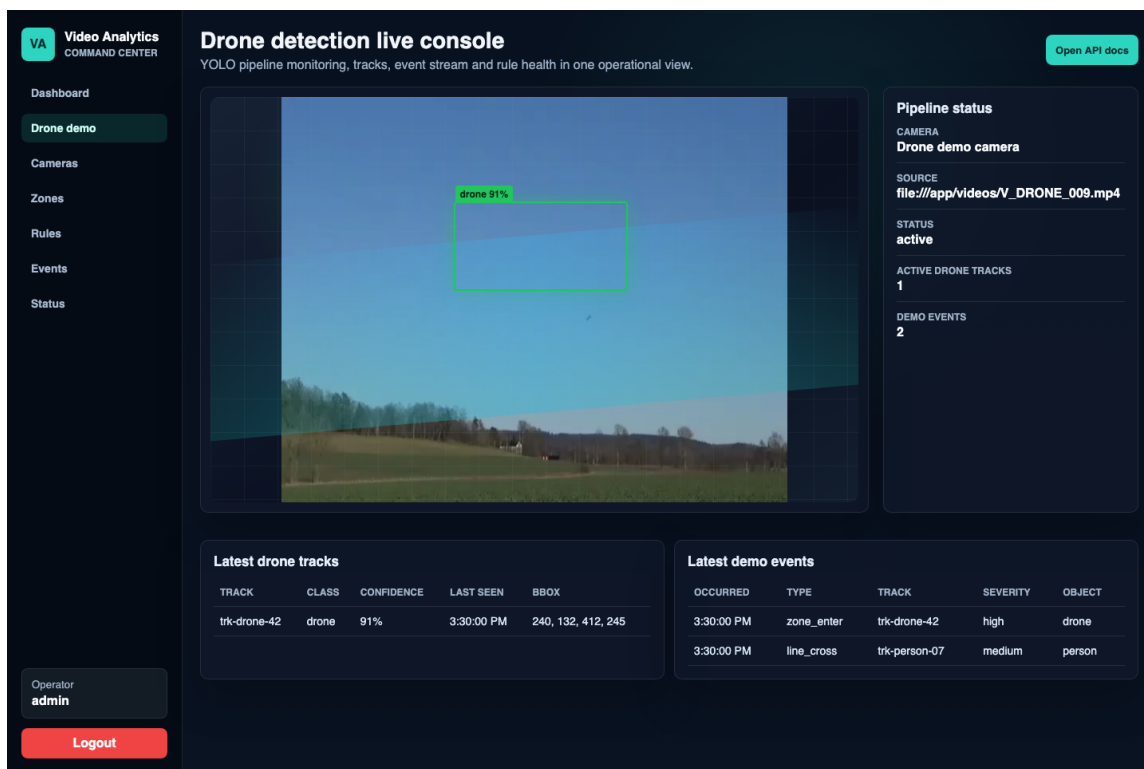


Рисунок 3.3 - Экран демонстрационного сценария обнаружения БПЛА

Экран статуса системы используется для быстрой проверки состояния, готовности и доступности инфраструктурных компонентов перед запуском приемочных сценариев (рисунок 3.4).

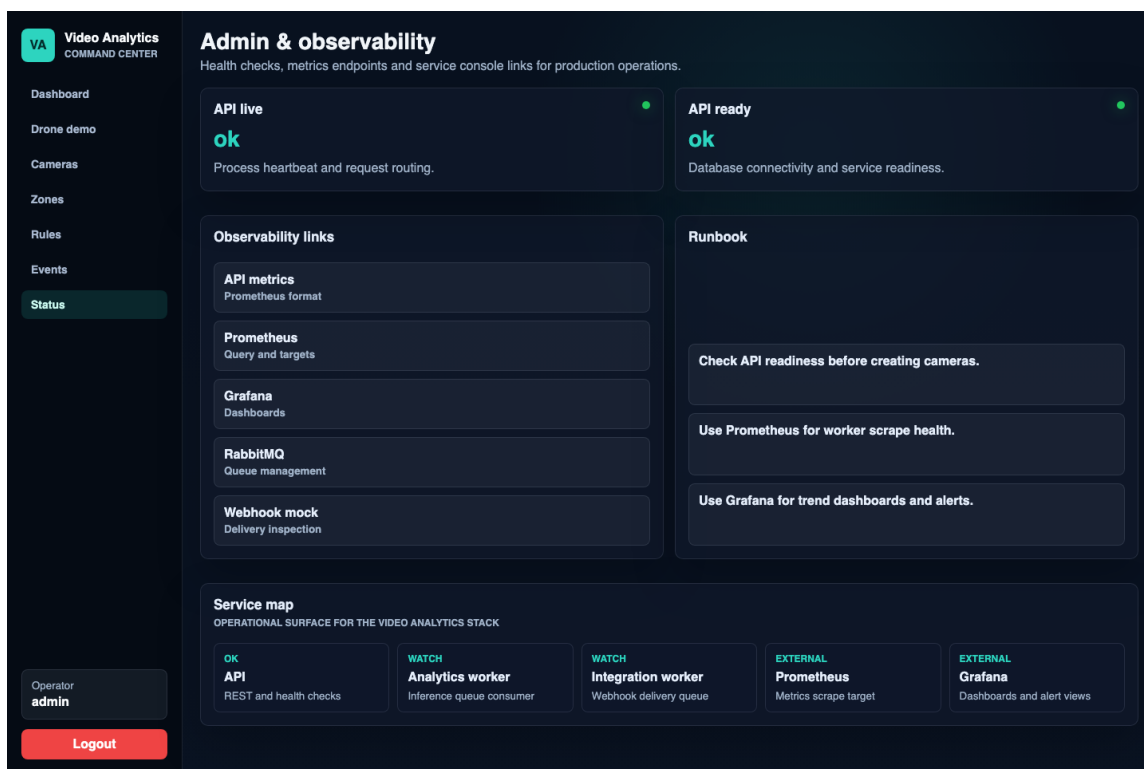


Рисунок 3.4 - Экран статуса сервисов и инфраструктуры

Журнал событий предоставляет фильтрацию по камере, типу события, уровню критичности и временному интервалу; действия оператора подключены к серверным операциям изменения состояния: подтверждение инцидента выполняется через `PATCH /api/v1/events/{event_id}/ack`, а экспорт доказательного JSON - через `GET /api/v1/events/{event_id}/evidence`. Пример интерфейса приведен на рисунке 3.5.

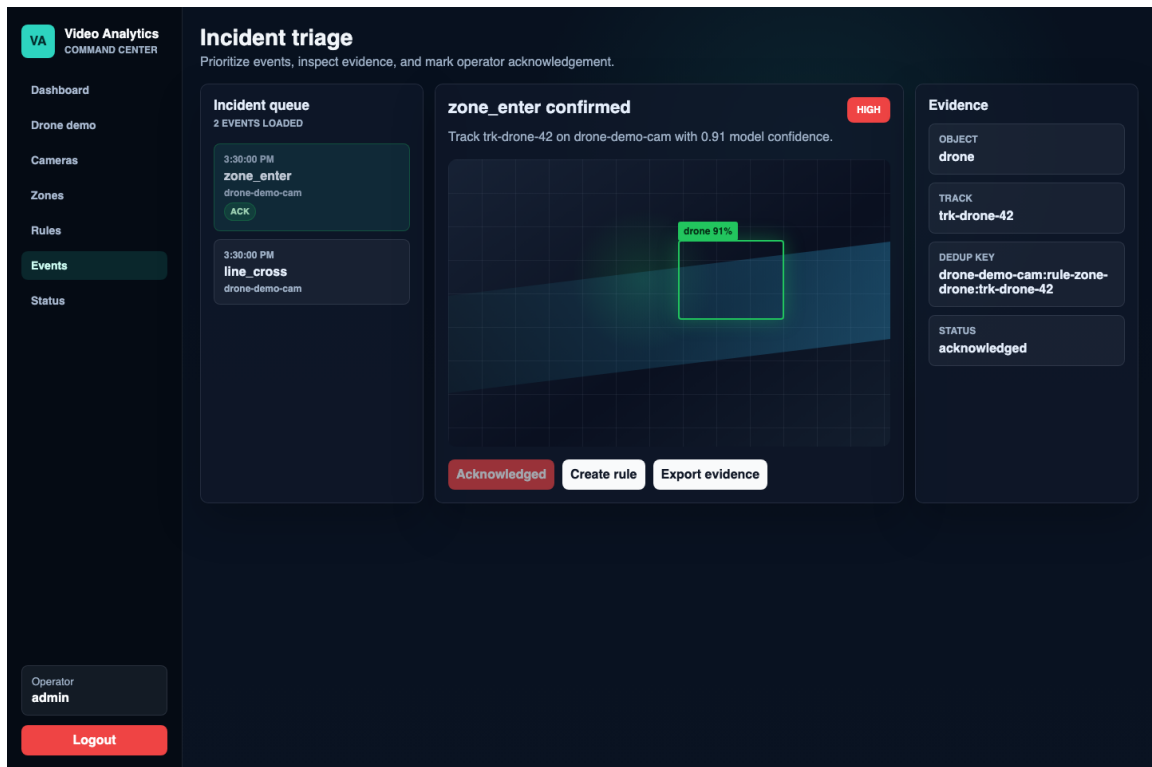


Рисунок 3.5 - Экран журнала событий видеоаналитики

3.8 Структура данных и миграции

3.8.1 Логическая схема БД

PostgreSQL содержит три группы сущностей:

- 1) Идентификация и доступ: `users`, `roles`, `user_roles`, `refresh_tokens`;
- 2) Конфигурация аналитики: `cameras`, `zones`, `rules`;
- 3) Исполнение и интеграция: `tracks`, `events`, `event_outbox`, `delivery_attempts`.

Таблица 3.3 - Ключевые инварианты данных

Инвариант	Назначение
events.event_id уникален	Гарантия адресуемости факта события
outbox.event_id FK на events	Связь записи исходящего сообщения с первичным фактом
events.dedup_key индексируется	Подавление дубликатов и быстрый поиск
events.acknowledged_at индексируется	Фиксация подтверждения инцидента оператором
refresh_tokens содержит только действующие сессии	Контролируемая ревокация токенов
delivery_attempts хранит каждую попытку доставки	Аудит надежности интеграционного контура

3.8.2 Подход к миграциям

Эволюция схемы БД выполняется через Alembic-миграции:

- каждая миграция атомарно описывает изменение схемы;
- миграции версионизируются и применяются в CI/стендах одинаково;
- откат (rollback) на один шаг доступен для контролируемого возврата схемы.

Такой подход обеспечивает воспроизводимость экспериментального контура и снижает риск дрейфа схемы между окружениями.

3.8.3 Индексы и производительность запросов

Для обеспечения приемлемой задержки запросов в журнал событий используются индексы по:

(3)

```
(camera_id, occurred_at),
event_type, severity, dedup_key.
```

Для контура исходящих сообщений предусмотрен индекс по статусу и времени следующей попытки:

`(status,next_retry_at).` (4)

Это уменьшает стоимость пакетных выборок публикатора при росте объема событий.

3.9 Тестирование, верификация и критерии качества

3.9.1 Стратегия тестирования

Тестовый контур разбит на уровни:

- 1) модульные тесты: отдельные функции валидации, дедупликации, расчетов повторных попыток;
- 2) интеграционные тесты: проверка взаимодействия сервиса с PostgreSQL/RabbitMQ;
- 3) проверочный интеграционный сценарий: вход пользователя, создание конфигурации и проход синтетически зафиксированного события до HTTP-получателя;
- 4) тесты пользовательского интерфейса: компонентные тесты и проверочный сценарий через Playwright.

Тесты серверной части выполняются через `pytest/pytest-asyncio`, тесты интерфейса - через `vitest` и Playwright.

3.9.2 Обязательные проверки до стендового прогона

Перед запуском измерительных сценариев выполняется чек-лист:

- миграции применены и схема БД актуальна;
- проверки состояния и готовности всех сервисов возвращают `ready`;
- в RabbitMQ объявлены обменники, очереди и очередь недоставленных сообщений;
- `webhook-mock` принимает тестовые POST-запросы;
- панели Prometheus/Grafana доступны и записывают данные.

3.9.3 Критерии приемки реализации

Реализация считается готовой к апробации, если одновременно выполнены условия:

- 1) все обязательные маршруты API доступны и валидируют входные данные;
- 2) RBAC-политика соблюдается для ролей `admin/operator/viewer`;
- 3) в интеграционном сценарии синтетически зафиксированное событие проходит путь до HTTP-получателя;
- 4) при отключении RabbitMQ события сохраняются в таблице исходящих сообщений и публикуются после восстановления;
- 5) метрики по FPS, задержке, повторным попыткам и очереди недоставленных сообщений доступны в Prometheus.

3.10 Эксплуатационные регламенты и ограничения текущего контура

3.10.1 Операционный регламент

Для штатной эксплуатации на стенде принимаются следующие регламенты:

- ежедневная проверка статусов камер и доли отброшенных кадров;
- контроль накопления `event_outbox` (пороговые предупреждения);
- контроль динамики `delivery_attempts` и очереди недоставленных сообщений;
- ротация и отзыв пользовательских сессий при смене ролей;
- архивирование артефактов экспериментов и логов.

3.10.2 Известные ограничения

Текущая реализация осознанно ограничена:

- базовый охранный профиль ориентирован на классы `person` и `vehicle`; БПЛА вынесены в отдельный демонстрационный профиль с классами `bird` и `drone`;
- отсутствие межкамерной повторной идентификации (ReID);
- односерверный профиль без кластерной высокой доступности (HA);
- ограниченный набор правил: `zone_enter`, `line_cross`,

`dwell_time.`

Эти ограничения не препятствуют достижению целей текущего исследования, но фиксируются как область дальнейшего развития.

3.11 Сценарии межсервисного взаимодействия

3.11.1 Сценарий «создание конфигурации оператором»

Последовательность взаимодействий:

- 1) Оператор аутентифицируется в пользовательском интерфейсе.
- 2) Пользовательский интерфейс получает JWT и запрашивает профиль ролей через `/api/v1/auth/me`.
- 3) Оператор создает/изменяет камеру, зоны и правила.
- 4) API валидирует полезную нагрузку запроса и записывает данные в PostgreSQL.
- 5) `analytics-worker` на очередном цикле конфигурационного опроса подхватывает изменения.

В результате измененная конфигурация начинает применяться без перекомпиляции и без остановки всего контура.

3.11.2 Сценарий «событие от кадра до HTTP-получателя»

Сквозной путь:

- 1) Камера передает кадр в `analytics-worker`.
- 2) Детектор и модуль сопровождения формируют обновленное состояние объекта.
- 3) Правильной движок создает событие и запись в таблице исходящих сообщений в одной транзакции.
- 4) `integration-worker` публикует событие в RabbitMQ.
- 5) Потребитель очереди вызывает внешний HTTP-получатель.
- 6) Статус попытки фиксируется в `delivery_attempts`.

Эта последовательность является целевой для расширенных сквозных проверок и измерения $r_{95}^{\text{obj} \rightarrow \text{event}}$; в текущей апробации количественно проверяется интеграционный участок от записи события до доставки.

3.11.3 Сценарий «временная недоступность брокера»

- 1) RabbitMQ недоступен, публикация в обменник не подтверждается.
- 2) Записи таблицы исходящих сообщений остаются в статусах `pending/retry`.
- 3) `analytics-worker` продолжает формировать события и записывать их в БД.
- 4) После восстановления брокера публикатор постепенно обрабатывает накопленную очередь.

Требуемый результат - отсутствие потери событий и ограниченное время догона очереди.

3.12 Конфигурационное управление и версионирование контрактов

3.12.1 Версионирование API

Все публичные маршруты вынесены под префикс `/api/v1`. Изменения, нарушающие обратную совместимость, должны оформляться как новая старшая версия (`/api/v2`) с параллельной поддержкой миграционного периода.

3.12.2 Версионирование сообщения события

Полезная нагрузка сообщения RabbitMQ содержит поле `schema_version`. Используется `schema_version = 1`. При добавлении полей:

- обратно совместимые изменения допустимы в той же версии;
- несовместимые изменения требуют перехода на новую версию контракта и новый ключ маршрутизации.

Такой подход снижает риск разрыва интеграций при развитии платформы.

3.12.3 Управление конфигурацией правил

Конфигурация зон/правил хранится в БД и версионизируется на уровне записей:

- фиксируется время обновления;
- фиксируется пользователь-инициатор;
- поддерживается `enabled`-флаг для безопасного включения/выключения правила.

Это обеспечивает управляемую эволюцию логики событий без изменения кода сервисов.

3.13 Инженерные аспекты производительности

3.13.1 Горизонт и вертикаль масштабирования

На текущем этапе основным механизмом остается вертикальное масштабирование: CPU, RAM и ускоритель инференса. При этом архитектура допускает ограниченное горизонтальное разнесение:

- вынесение `analytics-worker` на выделенный узел;
- вынесение `integration-worker` на отдельный узел;
- сохранение общих контрактов API и сообщений.

3.13.2 Параметры настройки фоновых обработчиков

К критическим параметрам тюнинга относятся:

- размер пакета публикатора исходящих сообщений;
- интервал опроса таблицы исходящих сообщений;
- ограничение числа параллельных вызовов HTTP-получателей;
- целевой FPS на камеру;
- профиль входного разрешения детектора.

Эти параметры калибруются экспериментально и фиксируются для каждого профиля стенда.

3.13.3 Управление обратным давлением

Обратное давление реализуется на трех уровнях:

- 1) буфер последнего актуального кадра в аналитическом контуре;
- 2) ограничение размера пакета публикатора исходящих сообщений;
- 3) ограничение числа одновременных HTTP-доставок в `integration-worker`.

Комбинация указанных механизмов предотвращает каскадное накопление очередей при кратковременных всплесках нагрузки.

3.14 Операционная диагностика и сопровождение

3.14.1 План диагностики при деградации

При ухудшении p95-задержки рекомендуется последовательная диагностика:

- 1) проверить рост числа отброшенных кадров и входной FPS;
- 2) проверить задержку детекции и загрузку CPU/ускорителя;
- 3) проверить размер накопленной очереди исходящих сообщений и скорость публикации;
- 4) проверить время ответа HTTP-получателя.

Такой порядок позволяет быстро локализовать проблемный слой: компьютерное зрение, БД/брокер или внешнюю интеграцию.

3.14.2 Набор базовых предупреждений

Минимальный набор предупреждений:

- `object->event p95` выше порога в течение заданного окна;
- непрерывный рост `event_outbox pending`;
- ненулевой рост очереди недоставленных сообщений в штатном режиме;
- недоступность `/health/ready` любого критичного сервиса.

Срабатывание предупреждений сопровождается фиксацией инцидента и сбором диагностического пакета для последующего анализа.

Выводы по главе 3

В третьей главе представлена практическая реализация платформы как набора изолированных сервисов с единым инфраструктурным контуром. Показано, как математическая модель главы 2 трансформирована в конкретные программные компоненты: API с JWT/RBAC, аналитический фоновый обработчик с целевым конвейером реального времени и интеграционный фоновый обработчик с гарантиями надежной доставки через таблицу исходящих

сообщений, повторные попытки и очередь недоставленных сообщений. Реализация задает целевой видеоконтур и отдельно проверяемый интеграционный контур, что соответствует границам апробации в главе 4.

4 АПРОБАЦИЯ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Цели, стенд и границы апробации

Цель апробации состоит в проверке того, что разработанная платформа обеспечивает корректную фиксацию событий, соблюдение ролевой модели доступа и надежную доставку сведений о событиях при отказах интеграционного контура. Методика согласована с критериями, сформулированными в главах 1 и 2, а также с общей логикой испытаний интеллектуальной видеоаналитики в системах видеонаблюдения [4; 5].

Фактическая апробация выполнена в двух контурах:

- 1) автоматизированный приемочный прогон на CPU-профиле стенда Docker Compose;
- 2) демонстрационный профиль обнаружения БПЛА на заранее подготовленном видеофрагменте и обученных весах детектора.

Аппаратное ускорение и многокамерная нагрузка рассматриваются как направление дальнейшего развития. В настоящей редакции количественные результаты заявляются только для CPU-профиля приемки и надежной доставки событий, а демонстрационный БПЛА-сценарий используется для проверки связности видеоконтура, операторского интерфейса и журнала событий.

Для исключения завышения выводов в работе используются три уровня доказательности. Первый уровень -- автоматизированный приемочный прогон RUN-2026-05-13-A1; по нему формулируются количественные значения задержек восстановления, статусов доставки и результатов RBAC-проверок. Второй уровень -- демонстрационный видеосценарий БПЛА; он подтверждает включение обученной модели в платформу и прохождение результата до интерфейса, но не используется как статистическая оценка промышленной задержки. Третий уровень -- расчетные критерии для будущего многокамерного стенда; они задают методику дальнейших измерений, но не объявляются достигнутым результатом настоящей апробации. В таблицах главы слово «пройден» относится только к явно указанному контуру проверки.

Таблица 4.1 - Контуры апробации и использованные артефакты

Контур	Артефакт	Назначение
CPU-профиль приемки	JSON RUN-A1	Проверка интеграционной доставки, RBAC, отказов RabbitMQ и HTTP-получателя
БПЛА-профиль	видео БПЛА и best.pt	Демонстрация обнаружения БПЛА и прохождения события через платформу
Обучение YOLO-детектора	results.csv, графики и матрица ошибок	Фиксация качества предметно дообученного детектора
План масштаби- рования	приложение В	Направление дальнейшего развития без заявленных приемочных метрик

4.2 Экспериментальные данные и артефакты

Для демонстрационного БПЛА-профиля использован набор данных Small Detection в формате YOLO с двумя целевыми классами: `bird` и `drone`. Этот профиль является отдельной конфигурацией целевых классов и не противоречит базовому охранному профилю с классами `person` и `vehicle`, описанному в главе 2. Наличие специализированных наборов данных и слабоконтролируемых сценариев разметки важно именно для БПЛА-задач, поскольку объект часто занимает малую площадь кадра, меняет масштаб и наблюдается на неоднородном фоне [6].

Таблица 4.2 - Параметры набора данных и обучения детектора

Параметр	Значение
Классы	bird, drone
Обучающая выборка	3982 изображения
Валидационная выборка	973 изображения
Базовая модель	вариант yolo26n, файл начальных весов yolo26n.pt
Размер входа	640 пикселей
Число эпох	100
Размер пакета	16
Устройство обучения	Apple MPS
Итоговые веса	best.pt из каталога обучения small_detection

К финальной эпохе обучения достигнуты следующие валидационные показатели: $\text{precision} = 0.8865$, $\text{recall} = 0.8677$, $mAP_{50} = 0.8810$, $mAP_{50:95} = 0.4161$. Значения взяты из последней строки файла `results.csv`; график обучения и матрица ошибок используются для визуального контроля динамики обучения и распределения ошибок. Эти значения не трактуются как промышленная сертификация детектора, но подтверждают пригодность модели для демонстрационного контура и дальнейшего дообучения на объектовых сценах.

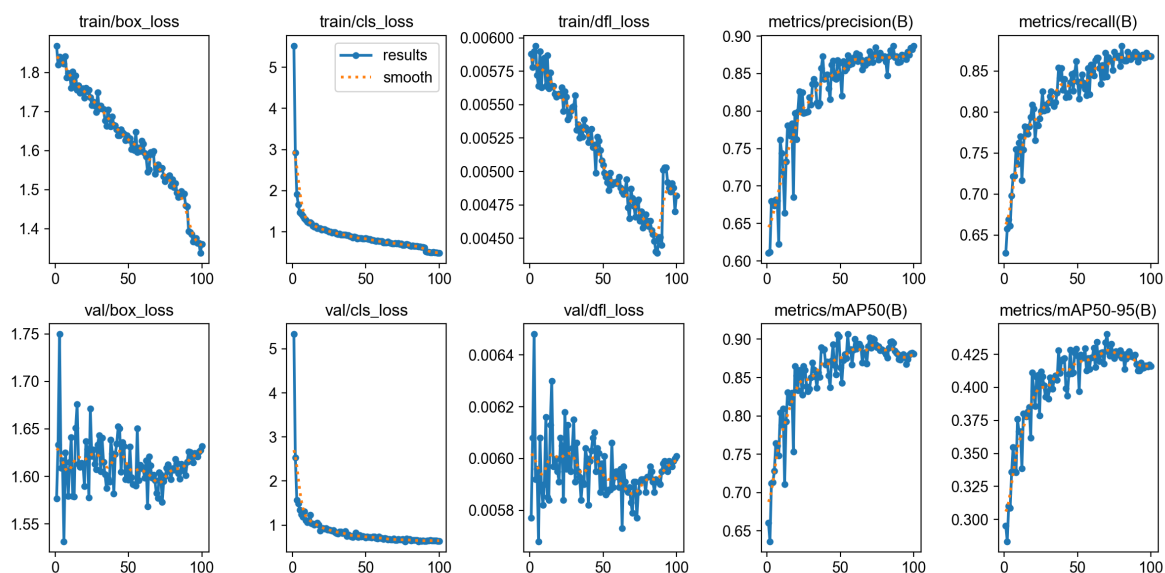


Рисунок 4.1 - Динамика метрик обучения детектора YOLO

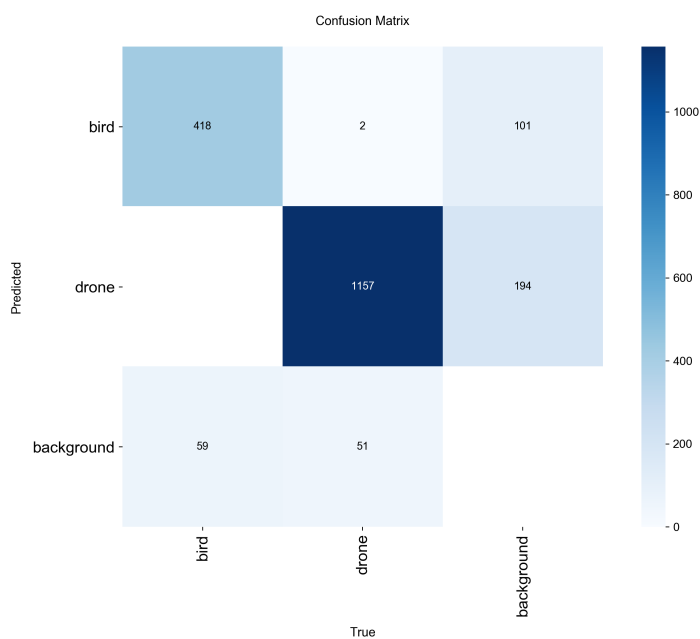


Рисунок 4.2 - Матрица ошибок детектора на валидационной выборке

Пример обнаружения БПЛА на демонстрационном видео приведен на рисунке 4.3. Данный артефакт включен в отчет не как отдельное сравнительное испытание, а как проверка связности прикладного сценария: обученные веса подключены к `analytics-worker`, а результат используется для формирования событийного потока платформы.



Рисунок 4.3 - Пример обнаружения БПЛА в демонстрационном видеофрагменте

4.3 Метрики и порядок расчета

Пусть Δt_j -- задержка от фиксации факта появления объекта до записи соответствующего события в таблицу исходящих сообщений для события j . Тогда расчетная p95-оценка для серии прогонов задается выражением

$$p95_{\text{obj} \rightarrow \text{event}} = \text{quantile}_{0.95}\{\Delta t_j\}. \quad (1)$$

Доля отброшенных кадров:

$$R_{\text{drop}} = \frac{N_{\text{dropped}}}{N_{\text{input}}}. \quad (2)$$

Доля успешной доставки во внешний контур:

$$R_{\text{delivery}} = \frac{N_{\text{delivered}}}{N_{\text{generated}}}. \quad (3)$$

Для отказных сценариев дополнительно фиксируются:

- статус записи исходящего сообщения во время отказа и после восстановления;
- число повторных попыток;
- количество успешных и неуспешных попыток доставки;
- время восстановления доставки после возврата внешнего компонента.

Для сценариев проверки безопасности критерием является соответствие фактического HTTP-статуса ожидаемому: например, попытка роли `viewer` выполнить операцию записи должна завершаться кодом 403.

Приемочный прогон считается успешным только при одновременном выполнении следующих условий: итоговое поле `result` в JSON-артефакте равно `passed`; запись исходящего сообщения для функционального сценария имеет статус `published`; счетчик успешных доставок функционального события равен единице при нулевом числе неуспешных доставок; при отказе RabbitMQ запись переходит в состояние `retry` и после восстановления получает статус `published`; при временной недоступности HTTP-получателя фиксируются неуспешные попытки и последующая успешная доставка; роль `viewer` получает запрет 403 при попытке выполнить операцию записи. Эти критерии проверяют не только наличие события, но и состояние транзакционной таблицы исходящих сообщений, транспортного контура и политики доступа.

4.4 Результаты приемочного прогона

Сохраненный приемочный артефакт `acceptance_results_2026-05-13.json` содержит результат `passed`; длительность автоматизированного прогона составила 30.732 с. В рамках сценария были созданы тестовые сущности камеры, зоны и правила, после чего проверены прохождение синтетически зафиксированного события `zone_enter` через интеграционный контур, отказ RabbitMQ, восстановление HTTP-получателя и запрет операции записи для роли `viewer`. Для событий `zone_enter`, `line_cross` и

dwel_l_time в данном прогоне проверялся не полный путь от видеокадра, а путь от записи события в базе данных до доставки внешнему HTTP-получателю.

Доказательная база приемочного прогона включает не только итоговый статус, но и контрольные поля артефакта: идентификаторы созданных камеры, зоны и правила; статусы записей таблицы исходящих сообщений до и после восстановления RabbitMQ; счетчики успешных и неуспешных доставок; время восстановления доставки; HTTP-статус 403 для запрещенной операции роли viewer. Таблицы ниже повторяют значения из JSON-артефакта и протоколов испытаний; они не являются самостоятельным источником измерений. Тем самым результат апробации связан с наблюдаемыми состояниями системы, а не с ручной оценкой интерфейса.

Таблица 4.3 - Функциональные результаты интеграционной проверки

Сценарий	Статус	Значение	Комментарий
zone_enter	пройден	1 событие	Синтетическое событие зафиксировано, запись исходящего сообщения переведена в published
line_cross	пройден	синт. событие	Проверен проход через таблицу исходящих сообщений и интеграционный контур
dwel_l_time	пройден	синт. событие	Проверен проход через контур повторной доставки HTTP-получателю
Интеграционная доставка	пройдена	success = 1, failed = 0	Базовое событие доставлено внешнему HTTP-получателю

Таблица 4.4 - Проверка надежности доставки при отказах

Сценарий	Потери	Время вос- становления	Комментарий
Отказ RabbitMQ	0	8.863 с	Во время отказа запись исходящего сообщения имела статус <code>retry</code> ; после восстановления статус стал <code>published</code>
Недоступность HTTP-получателя	0	4.455 с	Зафиксированы неуспешные попытки, затем успешная доставка после восстановления <code>webhook-mock</code>
Очередь недоставленных сообщений при исчерпании повторов	нет данных RUN-A1	не измерялось	Переход в очередь недоставленных сообщений проверен отдельным тестом логики фоновой обработчика; длительная недоступность HTTP-получателя требует отдельного стендового прогона

Таблица 4.5 - Результаты проверки безопасности в приемочном прогоне

Проверка	Результат	Интерпретация
Вход администратора	пройден	Маркеры доступа и обновления выданы, профиль роли доступен
viewer создает камеру	403	Запрет операции записи соответствует RBAC-матрице
Обновление и завершение сессии	пройден	Маркер обновления отзывается, повторное обновление запрещено
Валидация зон и правил	пройдена	Некорректные параметры полигона, линии и длительности отклоняются API

Таблица 4.6 - Производительность и эксплуатационные показатели

Показатель	Значение	Комментарий
Длительность приемочного прогона	30.732 с	Автоматизированный функционально-отказной прогон
Задержка функциональной доставки	4.605 с	Одинокое измерение интеграционной доставки; для р95 полного видеоконтура нужна серия длительных прогонов
Задержка восстановления RabbitMQ	8.863 с	Время до публикации после восстановления брокера
Задержка восстановления HTTP-получателя	4.455 с	Время до успешной доставки после восстановления HTTP-получателя
Число сообщений в очереди недоставленных	0 в RUN-A1	В штатном приемочном прогоне сообщений в очереди недоставленных не зафиксировано; сценарий исчерпания повторов проверен отдельно на уровне логики фонового обработчика
YOLO mAP_{50}	0.8810	Метрика автономной валидации демонстрационного детектора

4.5 Интерпретация результатов

Полученные результаты подтверждают достижение ключевых инженерных свойств платформы:

- функциональная цепочка «событие -- таблица исходящих сообщений -- RabbitMQ -- HTTP-получатель» работает для проверенных синтетиче-

ски зафиксированных событий;

- кратковременная недоступность RabbitMQ не приводит к потере событий, так как факт события сохраняется в БД до восстановления публикации;
- недоступность HTTP-получателя приводит к контролируемым неуспешным попыткам и последующей успешной доставке после восстановления;
- роль `viewer` не может изменять конфигурацию камер, что подтверждает применимость RBAC-политики для базового контура безопасности;
- предметно дообученный детектор демонстрирует достаточные автономно измеренные показатели для включения в демонстрационный контур БПЛА.

Вместе с тем количественная оценка `p95 object → event` требует не одиночного приемочного прогона, а серии длительных измерений при фиксированном числе камер, одинаковой частоте кадров и сохраненных временных рядах. Поэтому в настоящей работе `p95`-критерий оставлен как формальный приемочный критерий, а не как заявленное промышленное значение для всех профилей.

4.6 Ограничения апробации и угрозы валидности

Таблица 4.7 - Ограничения экспериментов и меры снижения риска

Ограничение	Влияние и мера контроля
Одиночный приемочный прогон	Позволяет проверить корректность сценариев, но не заменяет статистически устойчивую p95-оценку
Синтетические интеграционные события	Подтверждают путь доставки, но не измеряют полный вклад декодирования и инференса
Ограниченный набор БПЛА-сцен	Демонстрация требует расширения на реальные объектовые условия, разные фоны и масштабы
Нет длительной многокамерной серии	Не позволяет утверждать промышленные p95-границы; такие измерения вынесены в следующий стендовый этап
Очередь недоставленных сообщений не проверялась стендовым прогоном с исчерпанием повторов	Подтверждена логика перехода в очередь недоставленных сообщений, но не измерены длительность и операционные последствия длительного отказа HTTP-получателя
Односерверное развертывание	Проверяет базовую архитектуру, но не покрывает кластерную конфигурацию высокой доступности (HA) для PostgreSQL/RabbitMQ

4.7 Практические рекомендации по итогам апробации

По результатам финальной апробации сформулированы следующие рекомендации:

- 1) Для CPU-профиля необходимо ограничивать входной FPS и контролировать политику замещения последним актуальным кадром, чтобы задержка не накапливалась в очередях.

- 2) При аппаратном ускорении следует измерять не только задержку инференса, но и полный путь `object` → `event`, включая запись в БД и работу таблицы исходящих сообщений.
- 3) Отказные сценарии RabbitMQ и HTTP-получателя должны запускаться регулярно, так как они проверяют не алгоритм детекции, а сохранность доменного факта события.
- 4) Рост числа записей `event_outbox pending`, числа повторов и сообщений в очереди недоставленных должен сопровождаться предупреждениями Prometheus/Grafana и разбором причины деградации.
- 5) RBAC-проверки следует включать в обязательный регрессионный контроль при каждом изменении API-контрактов.

Выводы по главе 4

В четвертой главе проведена апробация разработанной платформы на CPU-профиле и демонстрационном БПЛА-сценарии. Подтверждены корректность интеграционной доставки синтетически зафиксированных событий, работоспособность механизма исходящих сообщений и повторной доставки при отказах RabbitMQ и HTTP-получателя, соблюдение RBAC-ограничений и пригодность дообученного детектора для демонстрационного контура. Результаты оформлены как воспроизводимый протокол стендового прогона с фиксированными критериями приемки, JSON-артефактом, таблицами метрик, чек-листами и актом. Отдельно указано, какие результаты подтверждены приемочным прогоном, какие -- модульными тестами, а какие являются демонстрационными материалами. Одновременно зафиксировано ограничение текущей проверки: промышленная p95-оценка полного видео-контура и стендовый сценарий исчерпания повторов требуют отдельной серии длительных прогонов с экспортом временных рядов и протоколов.

ЗАКЛЮЧЕНИЕ

В рамках магистерской работы разработана и экспериментально проверена платформа видеоаналитики реального времени для мониторинга охраняемых зон и событийного анализа. Основной результат работы состоит в создании контура «обнаружение -- сопровождение -- событийный анализ -- надежная доставка», в котором алгоритмическая обработка видеопотока связана с транзакционной фиксацией событий, асинхронной публикацией сведений о событиях внешним потребителям и проверяемой ролевой моделью доступа.

По результатам исследования получены следующие основные результаты:

- 1) Проанализирована предметная область охранного видеонаблюдения, рассмотрены существующие подходы к построению платформ видеоаналитики и сформулированы функциональные и нефункциональные требования к разрабатываемому решению.
- 2) Обоснован выбор методов обнаружения, сопровождения и правилowej генерации событий: одноэтапный детектор семейства YOLO, сопровождение по результатам обнаружения на основе SORT/DeepSORT и набор интерпретируемых событийных правил `zone_enter`, `line_cross`, `dwell_time`.
- 3) Построена формальная модель конвейера «захват кадра -- детекция -- сопровождение -- правила -- исходящие сообщения», включающая декомпозицию сквозной задержки, дедупликацию событий и критерии архитектурной приемки.
- 4) Реализована программная система в виде набора сервисов `api`, `analytics-worker`, `integration-worker`, `ui` с инфраструктурой PostgreSQL, RabbitMQ, Prometheus, Grafana и Docker Compose.
- 5) Проведена стендовая апробация: подтверждены интеграционная доставка синтетически зафиксированного события `zone_enter`, запрет операции записи для роли `viewer`, восстановление публикации после отказа RabbitMQ, повторная доставка после недоступности HTTP-получателя и связность демонстрационного БПЛА-видеосценария.

- 6) Определены ограничения текущей реализации и направления развития: длительная многокамерная оценка задержки, расширение набора классов, межкамерная аналитика, объектовое хранилище артефактов и промышленная отказоустойчивая конфигурация.

Практическая значимость работы заключается в том, что полученная платформа может использоваться как воспроизводимый исследовательско-экспериментальный стенд для задач охранного видеонаблюдения. Она позволяет проверять не только качество детектора, но и системные свойства, важные для эксплуатации: сохранность событий при отказах, управляемость повторной доставки, наблюдаемость и соблюдение ролевой модели.

Соответствие цели и задачам исследования подтверждается следующим:

- 1) первая задача выполнена: требования к платформе сформированы на основе анализа предметной области, существующих решений и ограничений охранного видеонаблюдения;
- 2) вторая задача выполнена: методы обнаружения, сопровождения и событийного анализа обоснованы сравнительным анализом и реализованы в программном контуре;
- 3) третья задача выполнена: модель, архитектура и алгоритмический конвейер платформы формализованы и связаны с требованиями к задержке, доставке событий и наблюдаемости;
- 4) четвертая задача выполнена: реализованы компоненты управления, аналитической обработки, интеграционной доставки, разграничения доступа, мониторинга и воспроизводимого развертывания;
- 5) пятая задача выполнена: стендовая апробация зафиксирована в виде протокола RUN-2026-05-13-A1, JSON-артефакта, чек-листов, акта стендового прогона, скриншотов, графиков обучения и таблиц метрик; сценарий перехода в очередь недоставленных сообщений подтвержден отдельной проверкой логики фоновой обработки; количественные выводы ограничены фактически выполненным приемочным контуром и не подменяют промышленную р95-оценку полного видеоконтура;
- 6) шестая задача выполнена: ограничения текущей реализации и направления перехода к промышленному применению явно сформулированы.

Апробация подтверждает выполнение цели именно на уровне исследовательско-экспериментального стенда: проверены фиксация и доставка событий, восстановление после кратковременных отказов интеграционного контура, соблюдение ролевых ограничений и связность демонстрационного видеосценария. Эти результаты достаточны для вывода о работоспособности архитектурно-алгоритмического подхода, но не являются заявкой на промышленную сертификацию платформы.

Ограничения работы также зафиксированы явно: текущая количественная апробация выполнена на CPU-профиле приемки для интеграционного контура, демонстрационный набор БПЛА подтверждает связность видеосценария, а промышленная многокамерная p95-оценка полного пути от объекта к событию требует отдельной длительной серии прогонов с экспортом временных рядов Prometheus/Grafana. Сценарий длительной недоступности HTTP-получателя до исчерпания повторов не измерялся в RUN-2026-05-13-A1 и отнесен к следующему стендовому этапу. Кроме того, текущая версия не включает межкамерную реидентификацию, кластерную отказоустойчивость PostgreSQL/RabbitMQ и промышленную калибровку детектора на объектовых сценах.

Дальнейшее развитие целесообразно вести по следующим направлениям:

- расширение набора поддерживаемых классов и дообучение детектора на предметных наборах данных конкретных охраняемых объектов;
- проведение длительной многокамерной серии прогонов для статистически устойчивой оценки $p95_{obj \rightarrow event}$;
- внедрение межкамерной корреляции и реидентификации;
- перенос снимков и видеоартефактов в объектное хранилище;
- кластеризация брокера и базы данных для эксплуатационного профиля высокой доступности;
- аппаратное ускорение инференса через ONNX Runtime или TensorRT для плотных многокамерных конфигураций.

Таким образом, поставленная цель достигнута на уровне исследовательско-экспериментального стенда: разработана, реализована и апробирована платформа видеоаналитики реального времени для мониторинга охраняемых зон и событийного анализа, обеспечивающая форми-

рование интерпретируемых событий, их сохранение и надежную доставку сведений о событиях внешним потребителям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. You Only Look Once: Unified, Real-Time Object Detection / J. Redmon [и др.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — 2016. — С. 779—788. — DOI: 10.1109/CVPR.2016.91.
2. *Wojke N., Bewley A., Paulus D.* Simple Online and Realtime Tracking with a Deep Association Metric // 2017 IEEE International Conference on Image Processing (ICIP). — 2017. — С. 3645—3649. — DOI: 10.1109/ICIP.2017.8296962.
3. *microservices.io*. Pattern: Transactional Outbox. — 2026. — URL: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обр. 30.04.2026).
4. IEC 62676-4:2025. Video surveillance systems for use in security applications -- Part 4: Application guidelines. — International Electrotechnical Commission, 2025. — URL: <https://webstore.iec.ch/en/publication/83425> (дата обр. 30.04.2026).
5. IEC 62676-6:2026. Video surveillance systems for use in security applications -- Part 6: Performance testing and grading of real-time intelligent video content analysis devices and systems for use in video surveillance applications. — International Electrotechnical Commission, 2026. — URL: <https://webstore.iec.ch/en/publication/59704> (дата обр. 30.04.2026).
6. *Chen Y.-H.* UAVDB: Point-Guided Masks for UAV Detection and Segmentation // arXiv preprint arXiv:2409.06490. — 2026. — DOI: 10.48550/arXiv.2409.06490. — URL: <https://arxiv.org/abs/2409.06490> (дата обр. 30.04.2026).
7. *Prometheus Authors*. Prometheus Documentation. — 2026. — URL: <https://prometheus.io/docs/> (дата обр. 30.04.2026).
8. *Kalman R. E.* A New Approach to Linear Filtering and Prediction Problems // Journal of Basic Engineering. — 1960. — Т. 82, № 1. — С. 35—45. — DOI: 10.1115/1.3662552.

9. *FFmpeg Project*. FFmpeg Documentation. — 2026. — URL: <https://ffmpeg.org/documentation.html> (дата обр. 30.04.2026).
10. *PyAV Developers*. PyAV Documentation. — 2026. — URL: <https://pyav.org/docs/stable/> (дата обр. 30.04.2026).
11. *Bernardin K., Stiefelhagen R.* Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics // *EURASIP Journal on Image and Video Processing*. — 2008. — Т. 2008. — С. 1—10. — DOI: 10.1155/2008/246309.
12. *MOTChallenge*. MOTChallenge Benchmark. — 2026. — URL: <https://motchallenge.net/> (дата обр. 30.04.2026).
13. *Richardson C.* *Microservices Patterns*. — Manning Publications, 2018. — ISBN 978-1617294549.
14. *RabbitMQ Team*. Dead Letter Exchanges. — 2026. — URL: <https://www.rabbitmq.com/dlx.html> (дата обр. 30.04.2026).
15. *Grafana Labs*. Grafana Documentation. — 2026. — URL: <https://grafana.com/docs/> (дата обр. 30.04.2026).
16. *National Institute of Standards and Technology*. Security and Privacy Controls for Information Systems and Organizations. — 2020. — DOI: 10.6028/NIST.SP.800-53r5. — URL: <https://csrc.nist.gov/Pubs/sp/800/53/r5/upd1/Final> (дата обр. 30.04.2026).
17. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Стандартинформ, 2017.
18. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. — Стандартинформ, 2008.
19. ГОСТ 7.1-2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Стандартинформ, 2003.
20. ГОСТ 7.80-2000. Библиографическая запись. Заголовок. Общие требования и правила составления. — Стандартинформ, 2000.

21. ГОСТ 7.82-2001. Библиографическая запись. Библиографическое описание электронных ресурсов. — Стандартинформ, 2001.
22. *FastAPI*. FastAPI Documentation. — 2026. — URL: <https://fastapi.tiangolo.com/> (дата обр. 30.04.2026).
23. *SQLAlchemy*. SQLAlchemy 2.0 Documentation. — 2026. — URL: <https://docs.sqlalchemy.org/> (дата обр. 30.04.2026).
24. *RabbitMQ Team*. RabbitMQ Documentation. — 2026. — URL: <https://www.rabbitmq.com/documentation.html> (дата обр. 30.04.2026).
25. *Docker*. Docker Documentation. — 2026. — URL: <https://docs.docker.com/> (дата обр. 30.04.2026).
26. Simple Online and Realtime Tracking / A. Bewley [и др.] // 2016 IEEE International Conference on Image Processing (ICIP). — 2016. — С. 3464—3468. — DOI: 10.1109/ICIP.2016.7533003.
27. *Redmon J., Farhadi A.* YOLOv3: An Incremental Improvement // arXiv preprint arXiv:1804.02767. — 2018. — URL: <https://arxiv.org/abs/1804.02767> (дата обр. 30.04.2026).
28. *Ultralytics*. Ultralytics YOLO Docs. — 2026. — URL: <https://docs.ultralytics.com/> (дата обр. 30.04.2026).
29. *Kuhn H. W.* The Hungarian Method for the Assignment Problem // Naval Research Logistics Quarterly. — 1955. — Т. 2, № 1/2. — С. 83—97. — DOI: 10.1002/nav.3800020109.
30. Microsoft COCO: Common Objects in Context / T.-Y. Lin, M. Maire, S. Belongie [и др.] // European Conference on Computer Vision. — 2014. — С. 740—755. — DOI: 10.1007/978-3-319-10602-1_48.
31. *RabbitMQ Team*. Publisher Confirms and Consumer Acknowledgements. — 2026. — URL: <https://www.rabbitmq.com/confirms.html> (дата обр. 30.04.2026).
32. *PostgreSQL Global Development Group*. PostgreSQL Documentation. — 2026. — URL: <https://www.postgresql.org/docs/> (дата обр. 30.04.2026).

33. *Macroscop*. Macroscop: профессиональное программное обеспечение для IP-камер. — 2026. — URL: <https://macroscop.com/> (дата обр. 20.05.2026).
34. *TRASSIR*. TRASSIR: цифровая платформа видеонаблюдения и видеоаналитики. — 2026. — URL: <https://trassir.ru/> (дата обр. 20.05.2026).
35. *AxxonSoft*. Axxon One Video Management Software. — 2026. — URL: https://www.axxonsoft.com/products/axxon_next/index.php (дата обр. 20.05.2026).
36. *Ivideon*. Ivideon Analytics. — 2026. — URL: <https://www.ivideon.com/analytics/> (дата обр. 20.05.2026).
37. *Milestone Systems*. XProtect Video Management Software. — 2026. — URL: <https://www.milestonesys.com/products/software/xprotect/> (дата обр. 20.05.2026).
38. *Vaidio*. Vaidio AI Vision Platform. — 2026. — URL: <https://www.vaidio.ai/platform> (дата обр. 20.05.2026).
39. *Alembic*. Alembic Documentation. — 2026. — URL: <https://alembic.sqlalchemy.org/> (дата обр. 30.04.2026).
40. *levan92*. deep-sort-realtime. — 2026. — URL: https://github.com/levan92/deep_sort_realtime (дата обр. 30.04.2026).

ПРИЛОЖЕНИЕ А. КОНТРАКТ СОБЫТИЯ И КОНЕЧНЫЕ ТОЧКИ API

А.1 Контракт события RabbitMQ

Версия схемы: schema_version=1.

```
{
  "schema_version": 1,
  "event_id": "uuid",
  "event_type": "zone_enter",
  "camera_id": "cam-01",
  "track_id": "trk-183",
  "object_class": "person",
  "confidence": 0.91,
  "occurred_at": "2026-03-07T10:15:32Z",
  "zone_id": "zone-perimeter-a",
  "rule_id": "rule-zone-enter-01",
  "severity": "high",
  "snapshot_path": "/data/artifacts/2026/03/07/event-123.jpg",
  "dedup_key": "cam-01:rule-zone-enter-01:trk-183",
  "acknowledged_at": null,
  "acknowledged_by": null,
  "attributes": {
    "direction": "inbound"
  }
}
```

А.2 Ключевые конечные точки REST API

POST/api/v1/auth/login	Получение пары access/refresh токенов
POST/api/v1/auth/refresh	Ротация токенов по refresh
POST/api/v1/auth/logout	Отзыв refresh-токена

GET/api/v1/auth/me	Профиль текущего пользователя и роли
GET/api/v1/users	Список пользователей (роль admin)
POST/api/v1/users	Создание пользователя и назначение ролей (admin)
POST/api/v1/cameras	Добавление камеры
GET/api/v1/cameras	Список камер и статусы
PATCH/api/v1/cameras/{camera_id}	Изменение параметров камеры
POST/api/v1/zones	Создание/обновление зоны контроля
POST/api/v1/rules	Создание/обновление правила
GET/api/v1/events	Просмотр журнала событий с фильтрами
PATCH/api/v1/events/{event_id}/ack	Подтверждение события оператором
GET/api/v1/events/{event_id}/evidence	Экспорт доказательного JSON по событию
GET/health/live	Проверка живости процесса
GET/health/ready	Проверка готовности зависимостей
GET/metrics	Экспорт метрик Prometheus

A.3 Пример dedup_key

Базовый формат ключа дедупликации:

$$\text{dedup_key} = \text{camera_id}:\text{rule_id}:\text{track_id}. \quad (4)$$

Для правил длительного нахождения допускается включение временного бакета:

$$\text{camera_id}:\text{rule_id}:\text{track_id}:\text{time_bucket}. \quad (5)$$

A.4 Примеры запросов и ответов API

Логин

```
POST /api/v1/auth/login
{
  "username": "operator1",
  "password": "*****"
```

```
}
```

200 OK

```
{  
  "access_token": "eyJhbGciOi...",  
  "refresh_token": "eyJhbGciOi...",  
  "token_type": "bearer",  
  "expires_in": 900  
}
```

Создание камеры

POST /api/v1/cameras

Authorization: Bearer <access_token>

```
{  
  "camera_id": "cam-01",  
  "name": "North Gate",  
  "source_uri": "rtsp://10.0.0.10/stream1",  
  "fps_target": 10,  
  "enabled": true  
}
```

201 Created

```
{  
  "camera_id": "cam-01",  
  "status": "active",  
  "created_at": "2026-05-13T07:49:00Z"  
}
```

Создание зоны

POST /api/v1/zones

Authorization: Bearer <access_token>

```
{  
  "zone_id": "zone-perimeter-a",  
  "camera_id": "cam-01",  
  "zone_type": "polygon",  
}
```

```

"geometry": {
  "type": "Polygon",
  "coordinates": [[
    [120,80],
    [620,80],
    [620,430],
    [120,430],
    [120,80]
  ]]
},
"name": "Perimeter A"
}

```

Создание правила

```

POST /api/v1/rules
Authorization: Bearer <access_token>
{
  "rule_id": "rule-zone-enter-01",
  "camera_id": "cam-01",
  "rule_type": "zone_enter",
  "severity": "high",
  "enabled": true,
  "params": {
    "zone_id": "zone-perimeter-a",
    "target_classes": ["person", "vehicle"]
  }
}

```

Чтение событий

```

GET /api/v1/events?
  camera_id=cam-01&event_type=zone_enter
Authorization: Bearer <access_token>

200 OK
[

```

```
{
  "event_id": "efbe8d6f-...",
  "event_type": "zone_enter",
  "camera_id": "cam-01",
  "track_id": "trk-183",
  "occurred_at": "2026-03-07T10:15:32Z",
  "severity": "high",
  "acknowledged_at": null,
  "acknowledged_by": null
}
]
```

Подтверждение события оператором

```
PATCH /api/v1/events/efbe8d6f-.../ack
Authorization: Bearer <access_token>
```

200 OK

```
{
  "event_id": "efbe8d6f-...",
  "acknowledged_at": "2026-05-12T09:10:00Z",
  "acknowledged_by": "admin"
}
```

Экспорт evidence

```
GET /api/v1/events/efbe8d6f-.../evidence
Authorization: Bearer <access_token>
Content-Disposition: attachment;
  filename="event-efbe8d6f-....json"
```

200 OK

```
{
  "event_id": "efbe8d6f-...",
  "status": "acknowledged",
  "snapshot_path": "/data/artifacts/2026/03/07/event-123.jpg",
}
```

```
    "exported_at": "2026-05-12T09:10:15Z"
}
```

A.5 Минимальная JSON-схема события (фрагмент)

```
{
  "type": "object",
  "required": [
    "schema_version",
    "event_id",
    "event_type",
    "camera_id",
    "track_id",
    "occurred_at",
    "severity",
    "dedup_key"
  ],
  "properties": {
    "schema_version": { "type": "integer", "const": 1 },
    "event_id": { "type": "string", "format": "uuid" },
    "event_type": { "type": "string" },
    "camera_id": { "type": "string" },
    "track_id": { "type": "string" },
    "occurred_at": { "type": "string", "format": "date-
time" },
    "severity": {
      "type": "string",
      "enum": ["low", "medium", "high"]
    },
    "dedup_key": { "type": "string" },
    "acknowledged_at": {
      "type": ["string", "null"],
      "format": "date-time"
    },
    "acknowledged_by": { "type": ["string", "null"] },
  }
}
```



```
    "attributes": { "type": "object" }  
  }  
}
```

ПРИЛОЖЕНИЕ Б. ТОПОЛОГИЯ РАЗВЕРТЫВАНИЯ И ПРОФИЛИ ЗАПУСКА

Б.1 Минимальный compose-контур

ui	SPA-интерфейс оператора
api	Управление конфигурацией, событиями, аутентификацией, RBAC и служебными маршрутами /health, /metrics
analytics-worker	Захват потоков, инференс, сопровождение объектов, правила, таблица исходящих сообщений
integration-worker	Публикация и доставка событий, повторы, очередь недоставленных сообщений
postgres	Хранение конфигурации, событий, таблицы исходящих сообщений и журнала доставок
rabbitmq	Асинхронный транспорт событий
prometheus	Сбор метрик
grafana	Визуализация метрик
webhook-mock	Тестовый HTTP-получатель

Б.2 Профили запуска

CPU-профиль разработки:

- 1--3 видеопотока;
- базовая легкая YOLO-модель на CPU;
- акцент на функциональные и интеграционные проверки.

Профиль дальнейшего аппаратного ускорения:

- 1--10 видеопотоков;
- инференс с аппаратным ускорением;
- фиксация p95-задержек и устойчивости доставки событий в отдельной серии прогонов.

Б.3 Операционные команды

запуск стека

make up-d

просмотр статуса

make ps

применение миграций

make migrate-up

запуск smoke-сценария

make smoke-e2e

остановка стека

make down

Б.4 Рекомендации по воспроизводимости

- 1) Фиксировать версии контейнерных образов и зависимостей.
- 2) Разделять базовую CPU-конфигурацию и профиль аппаратного ускорения через переменные окружения.
- 3) Сохранять артефакты прогонов: логи, snapshots Grafana, выгрузки метрик.
- 4) Проводить повторные прогоны при неизменной конфигурации для оценки разброса метрик.

Б.5 Переменные окружения (пример)

POSTGRES_DB	Имя базы данных платформы
POSTGRES_USER	Пользователь БД
POSTGRES_PASSWORD	Пароль пользователя БД
DATABASE_URL	Строка подключения API к PostgreSQL
RABBITMQ_URL	Строка подключения к RabbitMQ
JWT_SECRET	Секрет подписи JWT
JWT_ACCESS_TTL_MIN	Время жизни access-токена (минуты)

JWT_REFRESH_TTL_DAYS	Время жизни refresh-токена (дни)
WEBHOOK_TARGET_URL	URL целевого HTTP-получателя (в тестах - webhook-mock)
OUTBOX_BATCH_SIZE	Размер пакета публикации таблицы исходящих сообщений
OUTBOX_POLL_INTERVAL_MS	Интервал опроса таблицы исходящих сообщений
DELIVERY_MAX_RETRIES	Лимит повторов доставки
DELIVERY_BACKOFF_BASE_MS	Базовая задержка между повторами доставки
PROMETHEUS_ENABLED	Флаг экспорта метрик

Б.6 Фрагмент docker-compose (концептуально)

```

services:
  api:
    build: ./services/api
    depends_on: [postgres, rabbitmq]
  analytics-worker:
    build: ./services/analytics-worker
    depends_on: [postgres, rabbitmq]
  integration-worker:
    build: ./services/integration-worker
    depends_on: [postgres, rabbitmq, webhook-mock]
  ui:
    build: ./services/ui
    depends_on: [api]
  postgres:
    image: postgres:16
  rabbitmq:
    image: rabbitmq:3-management
  prometheus:

```

```
image: prom/prometheus
grafana:
  image: grafana/grafana
webhook-mock:
  image: mendhak/http-https-echo
```

Б.7 Профиль дальнейшего аппаратного ускорения

При переносе на стенд с аппаратным ускорением рекомендуется:

- 1) Включить профиль контейнеров с доступом к ускорителю.
- 2) Отдельно зафиксировать версию драйвера NVIDIA и среды исполнения.
- 3) Повторно откалибровать τ_{conf} , размер входа модели и целевой FPS.
- 4) Выполнить серию прогонов 1, 3, 5, 10 камер с неизменными правилами.
- 5) Для каждого шага мощности зафиксировать p95 object->event, dropped frames и delivery failures.

Эти шаги необходимы для сопоставимости профилей запуска и для корректной интерпретации масштабируемости платформы.

ПРИЛОЖЕНИЕ В. ПРОТОКОЛЫ ЭКСПЕРИМЕНТОВ

В.1 Карточка приемочного прогона

ID прогона	RUN-2026-05-13-A1
Дата и время	2026-05-13 07:49 UTC
Профиль стенда	приемка на CPU-профиле, Docker Compose
Команда повторения	make e2e-acceptance-artifact
Версия кода	рабочая копия проекта, артефакт acceptance_results_2026-05-13.json
Конфигурация камер	синтетическая камера, 1280x720, fps_target=10
Набор активных правил	zone_enter; для line_cross и dwell_time использованы синтетически зафиксированные события интеграционного контура
Длительность прогона	30.732 с
Сводный результат	ПРОЙДЕН

Критерии успешности RUN-2026-05-13-A1 фиксировались до анализа результатов: поле `result` должно иметь значение `passed`; функциональное событие должно быть опубликовано и доставлено; при отказе RabbitMQ запись исходящего сообщения должна сохраниться и перейти в состояние `published` после восстановления брокера; при недоступности HTTP-получателя должна быть зафиксирована повторная доставка; операция записи для роли `viewer` должна завершиться кодом 403. Промышленная p95-оценка полного видеоконтура в этот критерий не включалась и вынесена в отдельный будущий стендовый прогон.

Артефакты приложения интерпретируются отдельно. Измерительные артефакты содержат значения, которые прямо переносятся в таблицы главы 4: это JSON-файл приемочного прогона и файл `results.csv` обучения детектора. Проверочные артефакты подтверждают отдельные ветви логики, например переход сообщения в очередь недоставленных сообщений при ис-

черпаниии повторов. Демонстрационные артефакты, включая видео БПЛА и снимки интерфейса, подтверждают связность сценария и качество представления результата, но не используются для расчета p95-задержки полного видеоконтура.

В.2 Протокол метрик прогона

Таблица 4.12 - Протокол метрик RUN-2026-05-13-A1

Метрика	Значение	Источник данных
Задержка функциональной доставки	4.605 с	поле <code>functional_delivery</code> блока <code>timings_sec</code>
Задержка восстановления RabbitMQ	8.863 с	поле <code>rabbitmq_recovery</code> блока <code>timings_sec</code>
Задержка восстановления HTTP-получателя	4.455 с	поле <code>webhook_recovery</code> блока <code>timings_sec</code>
Повтор при отказе RabbitMQ	1	поле <code>retry_count</code> в блоке <code>rabbitmq_reliability</code>
Успешные доставки, функциональный сценарий	1	счетчик <code>success</code> в блоке <code>functional_e2e</code>
Неуспешные доставки, функциональный сценарий	0	счетчик <code>failed</code> в блоке <code>functional_e2e</code>
Неуспешные доставки, повтор HTTP-доставки	2	счетчик <code>failed</code> в блоке <code>webhook_retry</code>
Число сообщений в очереди недоставленных	0	приемочный артефакт; штатный прогон без конечной ошибки доставки
Статус создания камеры ролью viewer	403	проверка ограничения доступа

В.3 Протокол отказных сценариев

Сценарий отказа	Кратковременное отключение RabbitMQ при наличии события в таблице исходящих сообщений
Потеря событий	0
Статус во время отказа	retry, retry_count=1, published_at пустой
Статус после восстановления	published, retry_count=1, published_at=2026-05-13T07:49:52+00
Время восстановления	8.863 с
Состояние очереди недо- ставленных сообщений	сообщений не зафиксировано
Комментарий	Таблица исходящих сообщений сохранила событие до восстановления брокера; доставка завершилась без повторной генерации события аналитическим контуром
Сценарий отказа	Временная недоступность HTTP-получателя
Потеря событий	0
Неуспешные попытки до восстановления	1 зафиксированная попытка до успешной доставки; всего по событию: success = 1, failed = 2
Итоговый статус исходя- щего сообщения	published, published_at=2026-05-13T07:49:54+00
Время восстановления	4.455 с
Состояние очереди недо- ставленных сообщений	сообщений не зафиксировано
Комментарий	Политика повторной доставки позволила доставить событие после восстановления webhook-mock

В.4 Сохраненные артефакты

- JSON-артефакт приемочного прогона RUN-2026-05-13-A1: docs/MasterWork/REPORT/LaTeX/acceptance_results_2026-05-1

- 3.json;
- таблица метрик обучения: runs/detect/runs/train/small_detection/results.csv;
 - график обучения: runs/detect/runs/train/small_detection/results.png, включен в отчет как figures/yolo_training_results.png;
 - матрица ошибок: runs/detect/runs/train/small_detection/confusion_matrix.png, включена в отчет как figures/yolo_confusion_matrix.png;
 - итоговые веса детектора: runs/detect/runs/train/small_detection/weights/best.pt;
 - демонстрационное видео: videos/V_DRONE_009.mp4;
 - тест логики повторной доставки и очереди недоставленных сообщений: services/integration-worker/tests/test_consumer_retry.py;
 - скриншоты пользовательского интерфейса: figures/ui-dashboard.png, figures/ui-demo.png, figures/ui-status.png, figures/ui-events.png.

В.5 Чек-лист функциональных сценариев

Таблица 4.15 - Чек-лист функционального тестирования

ID	Сценарий	Ожидаемый результат	Статус
F-01	Вход с валидными учетными данными	Выдача маркеров доступа и обновления	ПРОЙДЕН
F-02	Обновление токена	Ротация маркера доступа без повторного входа	ПРОЙДЕН
F-03	Завершение сессии	Маркер обновления отозван, повторное обновление невозможно	ПРОЙДЕН
F-04	Создание камеры	Запись добавлена, камера доступна в списке	ПРОЙДЕН
F-05	Обновление параметров камеры	Изменения сохраняются, доступны через API и подхватываются фоновым обработчиком	ПРОЙДЕН
F-06	Создание зоны polygon	Геометрия валидирована и сохранена	ПРОЙДЕН
F-07	Создание правила zone_enter	Правило активно и доступно фоновому обработчику	ПРОЙДЕН
F-08	Сценарий zone_enter	Событие создано один раз и доставлено	ПРОЙДЕН
F-09	Сценарий line_cross	Событие проходит через таблицу исходящих сообщений и HTTP-получатель	ПРОЙДЕН, синтез.
F-10	Сценарий dwell_time	Событие проходит через контур повторной HTTP-доставки	ПРОЙДЕН, синтез.
F-11	Дедупликация по dedup_key	Повторное сообщение о событии не создает дубликат факта	ПРОЙДЕН, схема
F-12	Просмотр журнала событий	Фильтрация по камере, типу и времени поддерживается пользовательским интерфейсом и API	ПРОЙДЕН

В.6 Чек-лист надежности и безопасности

Таблица 4.16 - Чек-лист надежности и безопасности

ID	Сценарий	Ожидаемый результат	Статус
R-01	Отключение RabbitMQ при генерации событий	События остаются в таблице исходящих сообщений, потерь нет	ПРОЙДЕН
R-02	Восстановление RabbitMQ	Накопленная очередь исходящих сообщений обрабатывается, публикация возобновляется	ПРОЙДЕН
R-03	Ошибка HTTP-получателя 5xx/недоступность	Срабатывает политика повторной доставки	ПРОЙДЕН
R-04	Исчерпание лимита повторов	Сообщение попадает в очередь недоставленных	ЧАСТИЧНО, тест логики
R-05	Недоступность одной камеры	Другие камеры продолжают обработку	СЛЕДУЮЩИЙ СТЕНД
S-01	Роль viewer пытается изменить камеру	Доступ запрещен (403)	ПРОЙДЕН
S-02	Роль operator создает/меняет правила	Операция разрешена	ПРОЙДЕН
S-03	Роль admin управляет пользователями	Операция разрешена	ПРОЙДЕН
S-04	Запрос без JWT	Доступ к защищенному API запрещен	ПРОЙДЕН

В.7 Итоговый акт стендового прогона

Номер акта	АКТ-RUN-2026-05-13-A1
Дата и время	2026-05-13 07:49 UTC
Конфигурация стенда	приемка на CPU-профиле, Docker Compose, PostgreSQL, RabbitMQ, API, analytics-worker, integration-worker, webhook-mock
Номер версии	рабочая копия проекта, приемочный артефакт сохранен в каталоге отчета
Набор сценариев	Интеграционная доставка, отказ и восстановление RabbitMQ, повторная HTTP-доставка, RBAC-ограничение для роли viewer
Сводный результат	ПРОЙДЕН
Зафиксированные отклонения	промышленная р95-оценка для многокамерного профиля не выполнялась; требуется отдельная длительная серия прогонов
Решение по приемке	Платформа принята как воспроизводимый исследовательско-экспериментальный стенд текущего этапа
Ответственный исполнитель	Нурмухамедов А. Б.